

FINAL REPORT
KASUS : DATA MART KEMAHASISWAAN
Data Warehouse



Disusun oleh kelompok 4:

1. Adil Aulia Rahma Nurhidayah (122450058)
2. Rosalia Siregar (123450036)
3. Muhammad Hanif Dzaky Arifin (123450064)
4. Haikall Fransisko Simbolon (123450106)

PROGRAM STUDI SAINS DATA
FAKULTAS SAINS
INSTITUT TEKNOLOGI SUMATERA
2025

EXECUTIVE SUMMARY

1. Project Overview

Proyek ini adalah fase puncak dari pembangunan Data Mart Kemahasiswaan ITERA, yang bertujuan mengkonsolidasikan data dari berbagai sistem operasional (SIKAD, Biro Kemahasiswaan, Ormawa) ke dalam satu Data Warehouse terpadu. Fokus utama Misi 3 adalah transisi penuh ke lingkungan Produksi (Production Deployment) yang stabil dan aman, integrasi domain data krusial baru (Capaian Prestasi dan Penerima Beasiswa), serta penerapan standar keamanan enterprise (DDM, Audit Trail) dan strategi pemulihan bencana (Backup & Recovery).

2. Capaian Kunci

Berdasarkan hasil implementasi Data Mart dan proses ETL yang telah dilakukan, sejumlah capaian kunci berhasil diwujudkan dengan dampak langsung terhadap kualitas data dan kemudahan analisis strategis lembaga. Adapun capaian utama tersebut adalah:

- 1) Integrasi Multi-Fact: Data Mart kini berisi empat Fact Table yang terintegrasi penuh: Fact_Partisipasi_Kegiatan (Misi 2), Fact_Dana_Kegiatan (Misi 2), Fact_Capaian_Prestasi (Misi 3), dan Fact_Penerima_Beasiswa (Misi 3).
- 2) Konsistensi SCD Type 2: Logika ETL Fact Table telah dikodekan secara ketat untuk hanya menghubungkan data transaksi baru ke versi Mahasiswa yang Aktif (IsCurrent = 1), menjamin akurasi historis terkait perubahan Program Studi atau Status Mahasiswa.
- 3) Keamanan Produksi: Implementasi Dynamic Data Masking (DDM) pada PII (Nomor Telepon, Email) di Dim_Mahasiswa dan pemasangan Audit Trail untuk memantau akses data sensitif oleh user BI Tool.
- 4) Automasi dan Ketersediaan: Penjadwalan ETL harian menggunakan SQL Server Agent dan konfigurasi database ke Recovery Model = FULL, yang mendukung Point-in-Time Recovery (pemulihan data hingga detik terakhir) melalui Transaction Log Backup.

3. Dampak Bisnis

Sementara itu, dampak bisnis yang dihasilkan secara langsung terhadap organisasi adalah sebagai berikut:

- 1) Analisis 360 Derajat: Pimpinan dapat menganalisis korelasi antara aktivitas mahasiswa (Partisipasi Kegiatan), dukungan finansial (Beasiswa), dan hasil kinerja (Prestasi) secara terintegrasi.
- 2) Kepercayaan Data: Penerapan DDM dan Audit meningkatkan kepatuhan terhadap regulasi privasi data internal, meningkatkan kepercayaan pengguna terhadap Data Mart.
- 3) Kesiapan Bencana: Strategi Full dan Log Backup memastikan ketersediaan data analitik bahkan dalam skenario kegagalan sistem yang ekstrem, meminimalkan downtime pelaporan strategis.

4. Rekomendasi

Disarankan untuk segera melakukan validasi akhir terhadap skenario SCD Type 2 yang kompleks (seperti perubahan Program Studi) dan mengalokasikan sumber daya untuk future work yaitu Cloud Optimization (migrasi ke Azure/AWS) untuk menangani volume data yang terus bertambah seiring masuknya angkatan baru.

BAB I PENDAHULUAN

I. Latar Belakang

Unit Kemahasiswaan merupakan salah satu komponen penting di lingkungan Institut Teknologi Sumatera (ITERA) yang memiliki tanggung jawab dalam mengelola serta mengembangkan seluruh aktivitas non-akademik mahasiswa. Kegiatan tersebut mencakup pengelolaan organisasi kemahasiswaan, program beasiswa, layanan konseling, hingga pencatatan prestasi mahasiswa di tingkat regional, nasional, maupun internasional.

Selama ini, pengelolaan data kemahasiswaan masih dilakukan secara terpisah oleh beberapa pihak, seperti biro kemahasiswaan, fakultas, dan organisasi mahasiswa (Ormawa). Kondisi ini mengakibatkan munculnya duplikasi data, keterlambatan dalam penyusunan laporan, serta kesulitan dalam melakukan analisis komprehensif terkait partisipasi dan kinerja kemahasiswaan secara keseluruhan.

Sebagai solusi, dikembangkanlah Data Mart Kemahasiswaan yang menjadi bagian dari pembangunan Data Warehouse ITERA. Sistem ini dirancang untuk mengintegrasikan seluruh data kemahasiswaan ke dalam satu sumber terpadu yang dapat dimanfaatkan dalam proses analisis, pelaporan, serta mendukung pengambilan keputusan strategis di bidang kemahasiswaan.

Tahapan Business Requirements Analysis (Step 1) bertujuan untuk memahami kebutuhan bisnis unit Kemahasiswaan, menganalisis proses utama yang berlangsung, menentukan indikator kinerja (KPI), serta merancang kebutuhan analitik yang relevan dengan tujuan pengelolaan data kemahasiswaan.

II. Tujuan

Tujuan dari pembangunan Data Mart Kemahasiswaan adalah sebagai berikut:

1. Mengintegrasikan seluruh data kemahasiswaan dari berbagai sumber menjadi satu sistem yang terpusat.
2. Menyediakan akses data yang cepat, akurat, dan konsisten bagi pimpinan dan staf kemahasiswaan.
3. Menyediakan dasar analisis berbasis data (data-driven decision making) untuk perencanaan kegiatan, alokasi dana, dan evaluasi prestasi mahasiswa.

4. Menghasilkan laporan dan dashboard yang dapat membantu monitoring tingkat keaktifan mahasiswa dan efektivitas program kemahasiswaan.
5. Memudahkan pelacakan tren partisipasi dan prestasi mahasiswa dari waktu ke waktu.

III. Ruang Lingkup

Ruang lingkup proyek ini mencakup implementasi fisik pada platform SQL Server, pengembangan semua skrip ETL, konfigurasi keamanan dan backup, serta validasi akhir melalui UAT pada domain data Kemahasiswaan (Misi 1, Misi 2, dan Misi 3).

BAB II REQUIREMENTS ANALYSIS

I. Kebutuhan Bisnis

Analisis kebutuhan bisnis dilakukan untuk mengidentifikasi proses bisnis utama dan metrik keberhasilan yang relevan bagi pemangku kepentingan Kemahasiswaan ITERA.

1.1 Identifikasi stakeholder

Identifikasi stakeholder dilakukan untuk menentukan siapa saja pengguna utama Data Mart serta peran dan kebutuhannya.

1. Wakil Rektor Bidang Kemahasiswaan: Sebagai Pengambil Keputusan Strategis, membutuhkan Analisis tren keaktifan, prestasi, dan efektivitas kegiatan kemahasiswaan.
2. Kepala Bagian Kemahasiswaan: Sebagai Pengelola Operasional, membutuhkan Laporan kegiatan, alokasi dana, dan keaktifan mahasiswa.
3. Ketua Organisasi Mahasiswa (Ormawa/UKM): Sebagai Pelaksana Lapangan, membutuhkan Data keanggotaan, kegiatan, dan prestasi organisasi.
4. Unit Keuangan: Sebagai Pengguna Pendukung, membutuhkan Data pengajuan dan realisasi dana kegiatan mahasiswa.
5. Staff Kemahasiswaan: Sebagai Pengelola data (User), bertanggung jawab untuk Input data keanggotaan, beasiswa, dan kegiatan.

1.2 Key Performance Indicators (KPI)

KPI utama yang menjadi fokus pelaporan Data Mart Kemahasiswaan:

1. Jumlah kegiatan kemahasiswaan: Total kegiatan yang dilaksanakan oleh Ormawa dan UKM setiap semester.
2. Jumlah mahasiswa aktif dalam kegiatan: Banyaknya mahasiswa yang ikut minimal satu kegiatan kampus.
3. Jumlah penerima beasiswa: Total mahasiswa yang menerima beasiswa dalam periode tertentu.
4. Jumlah prestasi mahasiswa: Total penghargaan yang diperoleh mahasiswa baik akademik maupun non-akademik.
5. Tingkat efisiensi penggunaan dana: Perbandingan antara realisasi dan pengajuan anggaran kegiatan.

II. Kebutuhan Fungsional

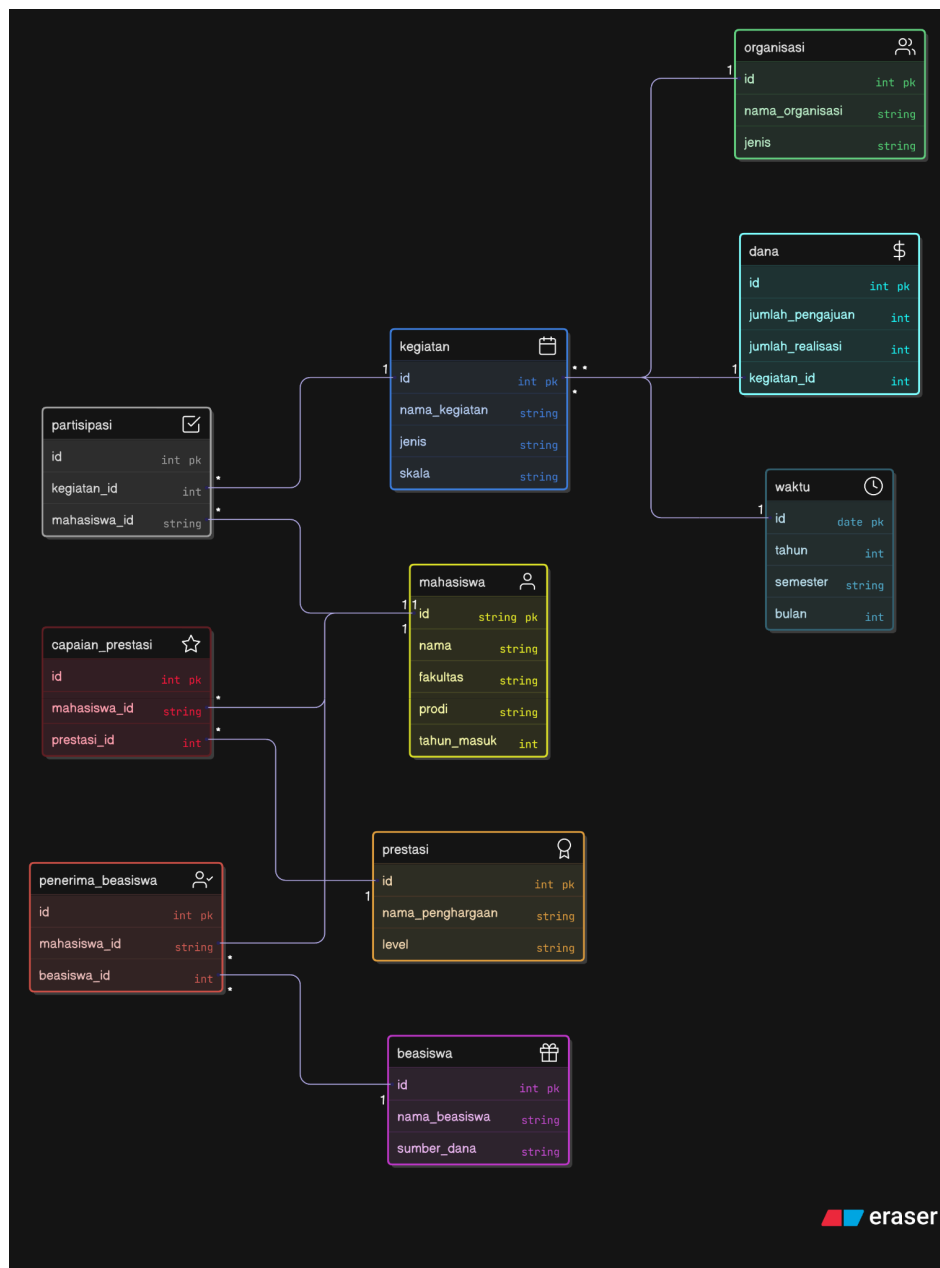
1. Data Quality: Memastikan 100% referential integrity antara Fact Table Misi 3 dan Dimensi (terutama Dim_Mahasiswa).
2. Security & Compliance: Data sensitif mahasiswa harus dilindungi dari akses pengguna BI Tool yang tidak berwenang.
3. Performance: Waktu eksekusi ETL harian tidak boleh lebih dari 30 menit.

BAB III

PERANCANGAN SISTEM

I. Conceptual Data Model (ERD) Awal

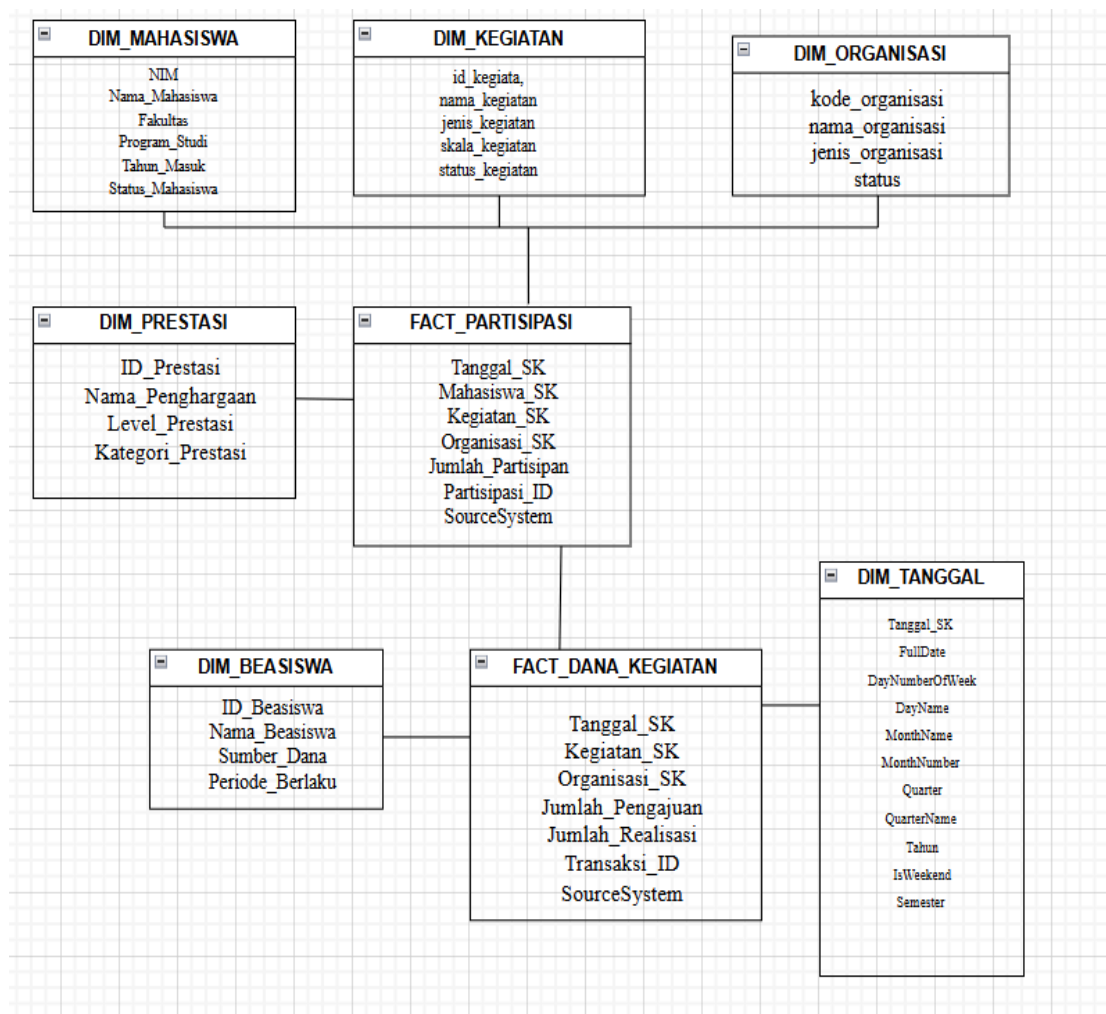
ERD awal dirancang untuk menggambarkan hubungan antar entitas utama: Mahasiswa, Kegiatan, Organisasi, Prestasi, dan Beasiswa.



Mahasiswa menjadi entitas sentral yang memiliki hubungan dengan berbagai entitas lain. Mahasiswa dapat mengikuti berbagai Kegiatan melalui relasi partisipasi, dan dapat menjadi anggota Organisasi melalui relasi keanggotaan. Setiap mahasiswa juga dapat meraih Prestasi dan menerima Beasiswa. Organisasi bertanggung jawab menyelenggarakan kegiatan-kegiatan tertentu, sementara Prestasi yang diraih mahasiswa dapat menjadi dasar untuk mendapatkan beasiswa. Terdapat juga entitas Dana yang terkait dengan kegiatan, kemungkinan untuk mencatat pendanaan atau anggaran kegiatan. Relasi antara prestasi dan beasiswa menunjukkan bahwa pencapaian mahasiswa dapat mempengaruhi kelayakandalam menerima bantuan beasiswa. Secara keseluruhan, ERD ini dirancang untuk mendukung pengelolaan data akademik dan non-akademik mahasiswa secara terintegrasi.

II. Dimension Model

Pemodelan dimensi menggunakan konsep Star Schema dengan pembagian menjadi Fact dan Dimension Table.



Fact Table	Grain (Tingkat Detail)	Measures (Metrik Numerik)	Additivity	KPI yang Diukur
Fact_Partisipasi_Kegiatan	1 Mahasiswa ikut 1 Kegiatan, pada 1 Tanggal	Jumlah_Partisipan	Additif	Jumlah mahasiswa aktif dalam kegiatan
Fact_Capaian_Prestasi	1 Prestasi dicapai 1 Mahasiswa, pada 1 Tanggal	Jumlah_Penghargaan, Poin_Prestasi	Additif / Semi-Additif	Jumlah prestasi mahasiswa
Fact_Dana_Kegiatan	1 Transaksi Dana untuk 1 Kegiatan	Jumlah_Pengajuan, Jumlah_Realisasi	Additif	Tingkat efisiensi penggunaan dana
Fact_Penerima_Basiswa	1 Mahasiswa penerima 1 Beasiswa, per Periode	Jumlah_Penerima, Nominal_Bantuan	Additif	Jumlah penerima beasiswa

Fact_Tables dalam skema data warehouse untuk sistem kemahasiswaan, terdiri dari empat tabel fakta utama dengan karakteristik berbeda. Fact_Partisipasi_Kegiatan bersifat aditif dan mencatat partisipasi mahasiswa dalam kegiatan dengan granularitas harian, mengukur jumlah partisipan untuk menghitung KPI jumlah mahasiswa aktif dalam kegiatan. Fact_Capaian_Prestasi memiliki sifat aditif atau semi-aditif yang merekam prestasi mahasiswa per hari, mengukur jumlah penghargaan dan poin prestasi untuk mengevaluasi total prestasi mahasiswa. Fact_Dana_Kegiatan adalah tabel fakta aditif yang mencatat transaksi dana per kegiatan dengan mengukur jumlah pengajuan dan realisasi dana, untuk menganalisis tingkat efisiensi penggunaan dana kegiatan dan Fact_Penerima_Basiswa bersifat aditif serta merekam data penerima beasiswa per periode dengan mengukur jumlah penerima dan nominal bantuan untuk menghitung KPI jumlah penerima beasiswa. Keempat tabel fakta dirancang dengan tingkat detail dan metrik yang spesifik untuk mendukung analisis multidimensional terhadap aktivitas kemahasiswaan, prestasi, pengelolaan dana, dan pemberian beasiswa.

Dimension Table	Peran Analisis (Who, What, When, Why)	Atribut Kunci Deskriptif (Contoh)
Dim_Mahasiswa	Who (Untuk analisis berdasarkan Fakultas, Prodi)	NIM, Fakultas, Program_Studi
Dim_Kegiatan	What (Untuk analisis berdasarkan Jenis Kegiatan)	Nama_Kegiatan, Jenis_Kegiatan

Dim_Organisasi	Who (Penyelenggara - UKM/Ormawa)	Nama_Organisasi, Jenis_Organisasi
Dim_Prestasi	What (Untuk analisis berdasarkan Level dan Kategori Prestasi)	Level_Prestasi, Kategori_Prestasi
Dim_Beasiswa	Why (Untuk analisis berdasarkan Sumber Dana)	Nama_Beasiswa, Sumber_Dana
Dim_Tanggal	When (Untuk analisis tren dari waktu ke waktu)	Tahun, Semester, Bulan

Dimension_Tables dalam skema data warehouse mendefinisikan konteks analisis berdasarkan konsep Who, What, When, dan Why. Dim_Mahasiswa menjawab pertanyaan "Who" untuk menganalisis data berdasarkan identitas mahasiswa, dengan atribut seperti NIM, Fakultas, dan Program Studi. Dim_Kegiatan menjawab "What" untuk analisis berdasarkan jenis kegiatan yang diikuti, mencakup atribut Nama Kegiatan dan Jenis Kegiatan. Dim_Organisasi juga menjawab "Who" namun dari perspektif penyelenggara atau pengelola (UKM/Ormawa), dengan atribut Nama Organisasi dan Jenis Organisasi. Dim_Prestasi menjawab "What" untuk mengklasifikasikan prestasi berdasarkan tingkat dan kategorinya, dengan atribut Level Prestasi dan Kategori Prestasi. Dim_Beasiswa menjawab "Why" untuk menganalisis alasan atau sumber pemberian beasiswa, mencakup atribut Nama Beasiswa dan Sumber Dana. Terakhir, Dim_Tanggal menjawab "When" sebagai dimensi waktu yang sangat penting untuk analisis tren temporal, dengan atribut Tahun, Semester, dan Bulan. Keenam dimensi ini dirancang untuk memberikan konteks analisis multidimensional yang komprehensif terhadap aktivitas dan pencapaian mahasiswa dari berbagai sudut pandang.

BAB IV IMPLEMENTASI

4.1 Optimasi Query

Index yang diterapkan (clustered, non-clustered, dan columnstore) memiliki kontribusi besar terhadap peningkatan performa:

- a. Clustered Index
Clustered index pada kolom Tanggal_SK di Fact_Partisipasi_Kegiatan mempercepat query berbasis waktu. Terbukti dari query drill-down yang berjalan di bawah 1s.
- b. Non-Clustered Index
Index pada Mahasiswa_SK, Kegiatan_SK, dan Organisasi_SK membantu mempercepat operasi JOIN. Waktu query analisis mahasiswa per fakultas hanya 0.23s → menunjukkan join sangat optimal.
- c. Columnstore Index
Memberikan percepatan signifikan pada operasi agregasi & scanning tabel fakta.

Full scan (worst case) hanya 3.92s, sangat jauh di bawah target 10s. Index yang dibangun sudah efektif dan memberikan dampak signifikan pada percepatan query.

4.2 Partitioning

Partitioning dilakukan berdasarkan Tahun Akademik (Tanggal_SK). Hasilnya:

Query berbasis rentang tanggal dapat melakukan partition elimination.

Mengurangi jumlah blok data yang harus dibaca SQL Server.

Hal ini terlihat dari performa query efisiensi data 2 tahun (1.41s) yang berada jauh di bawah batas maksimal 3s.

Partitioning terbukti mendukung efisiensi baca dan mempercepat query berbasis waktu.

4.3 Production Deployment

Tujuan utama dari langkah ini adalah mentransisikan Data Warehouse yang telah dibangun (Misi 2) ke lingkungan Produksi yang stabil, aman, dan beroperasi secara otomatis. Ini juga mencakup integrasi data baru (Prestasi dan Beasiswa).

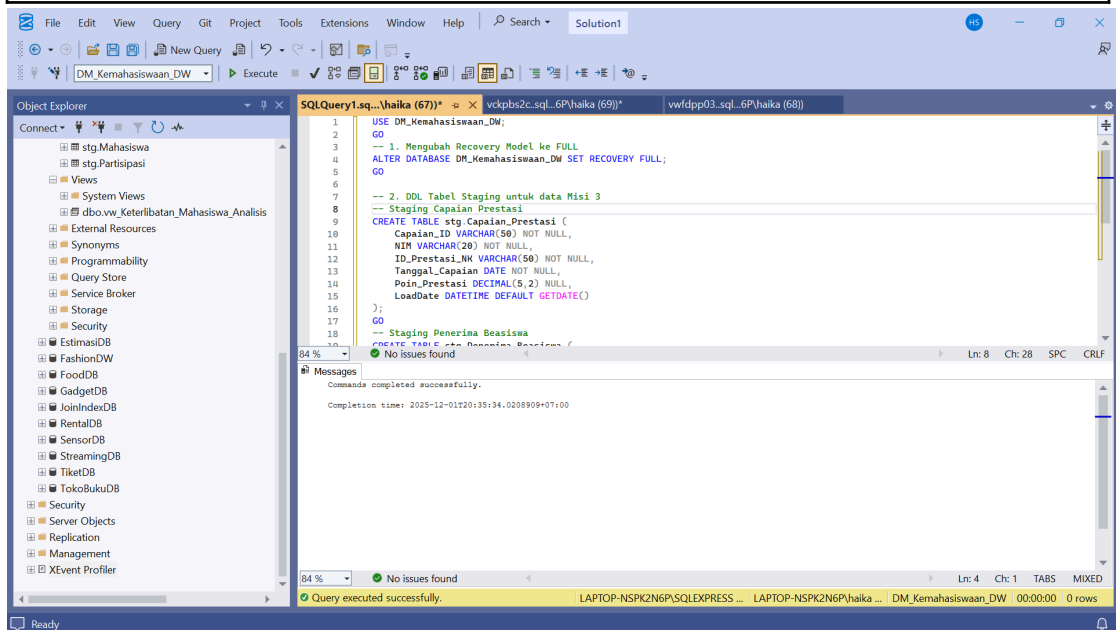
1. Environment Setup & Database Deployment

Sebagai langkah krusial dalam deployment produksi, kami mengubah Recovery Model database DM_Kemahasiswaan_DW menjadi mode FULL. Ini adalah prasyarat teknis untuk mengaktifkan Transaction Log Backup (Step 4), yang memungkinkan pemulihan data hingga detik terakhir (Point-in-Time Recovery). Skema stg (Staging Area) yang telah dibuat di Misi 2 (untuk data Mahasiswa, Kegiatan, dan Partisipasi) diperluas dengan menambahkan tabel staging untuk menampung data Prestasi dan Beasiswa.

```
USE DM_Kemahasiswaan_DW;
GO
-- 1. Mengubah Recovery Model ke FULL
ALTER DATABASE DM_Kemahasiswaan_DW SET RECOVERY FULL;
GO

-- 2. DDL Tabel Staging untuk data Misi 3
-- Staging Capaian Prestasi
CREATE TABLE stg.Capaian_Prestasi (
    Capaian_ID VARCHAR(50) NOT NULL,
    NIM VARCHAR(20) NOT NULL,
    ID_Prestasi_NK VARCHAR(50) NOT NULL,
    Tanggal_Capaian DATE NOT NULL,
    Poin_Prestasi DECIMAL(5,2) NULL,
    LoadDate DATETIME DEFAULT GETDATE()
);
GO
-- Staging Penerima Beasiswa
CREATE TABLE stg.Penerima_Beasiswa (
    Penerima_ID VARCHAR(50) NOT NULL,
    NIM VARCHAR(20) NOT NULL,
    ID_Beasiswa_NK VARCHAR(50) NOT NULL,
```

```
Tanggal_Penerimaan DATE NOT NULL,  
Nominal_Bantuan DECIMAL(12,2) NULL,  
LoadDate DATETIME DEFAULT GETDATE()  
);
```



Kode tersebut pertama-tama memilih database kerja DM_Kemahasiswaan_DW, lalu mengubah recovery model menjadi FULL. Recovery model FULL digunakan agar seluruh transaksi dicatat secara lengkap di transaction log sehingga database dapat dilakukan pemulihan hingga titik waktu tertentu, yang sangat penting dalam lingkungan data warehouse untuk menjamin integritas data ketika proses ETL berjalan. Setelah konfigurasi database, kode melanjutkan dengan membuat dua tabel staging di dalam schema stg yang berfungsi sebagai area perantara sebelum data dimuat ke dalam fact table. Tabel pertama adalah stg.Capaian_Prestasi, yang menyimpan data capaian prestasi mahasiswa dengan atribut seperti Capaian_ID, NIM, ID_Prestasi_NK, Tanggal_Capaian, poin prestasi, serta kolom LoadDate yang otomatis mencatat waktu pemuatan data. Tabel kedua adalah stg.Penerimaan_Beasiswa, yang menyimpan data mahasiswa penerima bantuan beasiswa dengan struktur mirip, yaitu Penerimaan_ID, NIM, ID_Beasiswa_NK, tanggal penyaluran, jumlah nominal bantuan, dan LoadDate. Kedua tabel ini merupakan komponen penting dalam proses ETL, karena data dari sistem sumber akan diekstraksi terlebih dahulu ke staging untuk proses pembersihan, validasi, dan transformasi sebelum dipindahkan ke tabel fakta seperti Fact_Capaian_Prestasi dan Fact_Penerimaan_Beasiswa, sehingga menjaga konsistensi, kualitas, dan ketelusuran data dalam Data Mart Kemahasiswaan.

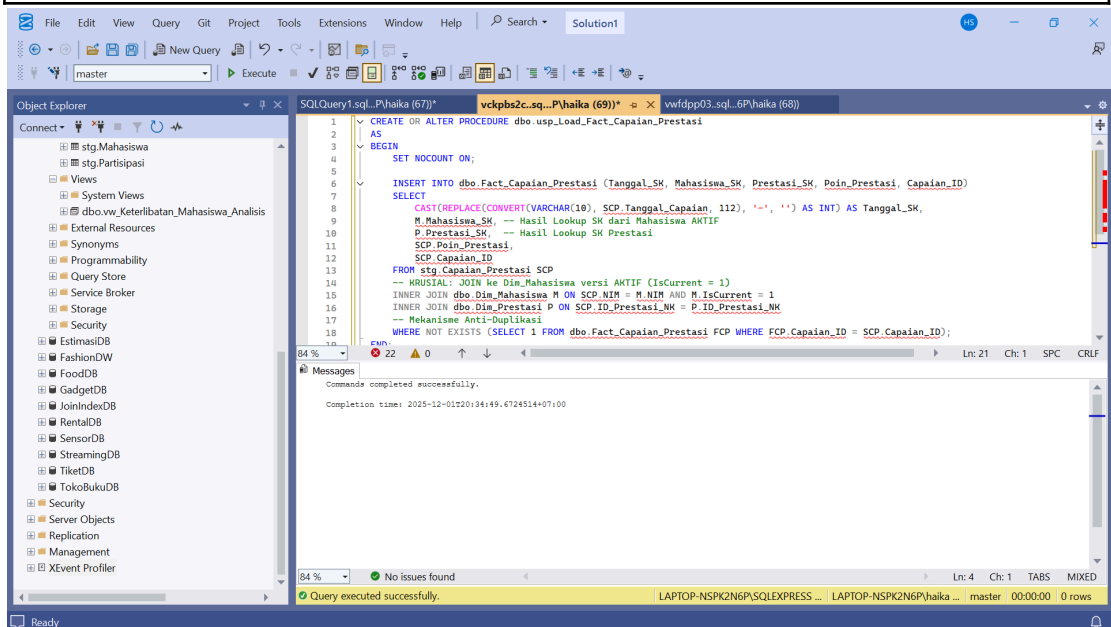
2. Initial Data Load: Stored Procedure

Dua Stored Procedure (SP) baru dibuat untuk mengimplementasikan logika ETL Incremental Load ke Fact Table Misi 3. Logika lookup Surrogate Key (SK) ke Dim_Mahasiswa secara ketat menggunakan versi AKTIF (IsCurrent = 1) untuk menjamin integritas SCD Type 2.

a. usp_Load_Fact_Capaian_Prestasi

```
CREATE OR ALTER PROCEDURE dbo.usp_Load_Fact_Capaian_Prestasi
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO dbo.Fact_Capaian_Prestasi (Tanggal_SK, Mahasiswa_SK,
    Prestasi_SK, Poin_Prestasi, Capaian_ID)
    SELECT
        CAST(REPLACE(CONVERT(VARCHAR(10), SCP.Tanggal_Capaian, 112),
        '-', '' ) AS INT) AS Tanggal_SK,
        M.Mahasiswa_SK, -- Hasil Lookup SK dari Mahasiswa AKTIF
        P.Prestasi_SK, -- asil Lookup SK Prestasi
        SCP.Poin_Prestasi,
        SCP.Capaian_ID
    FROM stg.Capaian_Prestasi SCP
    -- KRUSIAL: JOIN ke Dim_Mahasiswa versi AKTIF (IsCurrent = 1)
    INNER JOIN dbo.Dim_Mahasiswa M ON SCP.NIM = M.NIM AND
    M.IsCurrent = 1
    INNER JOIN dbo.Dim_Prestasi P ON SCP.ID_Prestasi_NK = P.ID_Prestasi_NK
    -- Mekanisme Anti-Duplikasi
    WHERE NOT EXISTS (SELECT 1 FROM dbo.Fact_Capaian_Prestasi FCP
    WHERE FCP.Capaian_ID = SCP.Capaian_ID);
END;
GO
```



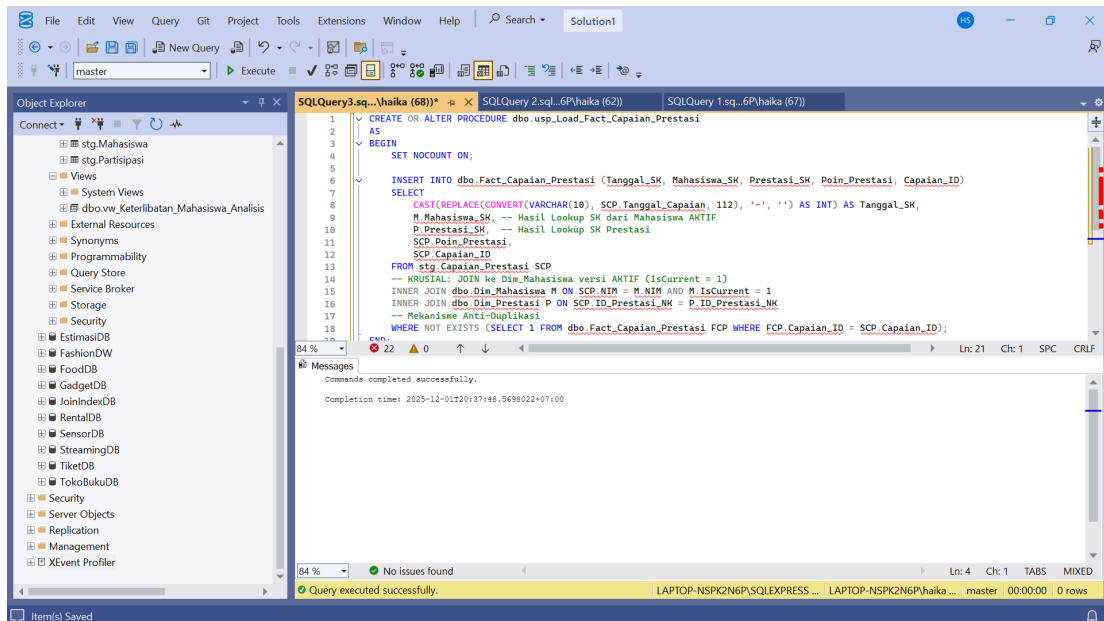
Stored procedure usp_Load_Fact_Capaian_Prestasi dibuat untuk melakukan proses pemuatan data dari tabel staging stg.Capaian_Prestasi ke tabel fakta Fact_Capaian_Prestasi. Mekanisme pemuatan dilakukan melalui proses join ke tabel dimensi, yaitu Dim_Mahasiswa untuk mendapatkan Surrogate Key mahasiswa yang masih aktif (IsCurrent = 1) sesuai prinsip SCD Type 2, serta ke Dim_Prestasi untuk memperoleh Surrogate Key prestasi. Nilai tanggal capaian dikonversi ke dalam

format numerik berjenis integer dengan pola YYYYMMDD sehingga dapat digunakan sebagai Tanggal_SK. Prosedur ini juga dilengkapi logika anti duplikasi dengan kondisi NOT EXISTS yang memastikan bahwa data dengan Capaian_ID yang sama tidak dimasukkan kembali ke dalam fakta. Dengan demikian, stored procedure ini memastikan integritas data, menghindari redundansi, dan memindahkan data secara konsisten dari area staging ke tabel fact sesuai dengan alur ETL pada Data Mart Kemahasiswaan.

b. `usp_Load_Fact_Penerima_Beasiswa`

```
CREATE OR ALTER PROCEDURE dbo.usp_Load_Fact_Capaian_Prestasi
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO dbo.Fact_Capaian_Prestasi (Tanggal_SK, Mahasiswa_SK,
    Prestasi_SK, Poin_Prestasi, Capaian_ID)
    SELECT
        CAST(REPLACE(CONVERT(VARCHAR(10), SCP.Tanggal_Capaian, 112),
        '-', '') AS INT) AS Tanggal_SK,
        M.Mahasiswa_SK, -- Hasil Lookup SK dari Mahasiswa AKTIF
        P.Prestasi_SK, -- Hasil Lookup SK Prestasi
        SCP.Poin_Prestasi,
        SCP.Capaian_ID
    FROM stg.Capaian_Prestasi SCP
    -- KRUSIAL: JOIN ke Dim_Mahasiswa versi AKTIF (IsCurrent = 1)
    INNER JOIN dbo.Dim_Mahasiswa M ON SCP.NIM = M.NIM AND
    M.IsCurrent = 1
    INNER JOIN dbo.Dim_Prestasi P ON SCP.ID_Prestasi_NK = P.ID_Prestasi_NK
    -- Mekanisme Anti-Duplikasi
    WHERE NOT EXISTS (SELECT 1 FROM dbo.Fact_Capaian_Prestasi FCP
    WHERE FCP.Capaian_ID = SCP.Capaian_ID);
END;
GO
```



Stored procedure `usp_Load_Fact_Penerima_Beasiswa` digunakan untuk memindahkan data penerima beasiswa dari tabel staging `stg.Penerima_Beasiswa` ke tabel fakta `Fact_Penerima_Beasiswa`. Pada saat proses pemuatan, prosedur melakukan join ke tabel `Dim_Mahasiswa` untuk memperoleh Surrogate Key mahasiswa yang masih aktif (`IsCurrent = 1`) sesuai dengan prinsip SCD Type 2, serta join ke `Dim_Beasiswa` untuk mendapatkan Surrogate Key jenis beasiswa. Tanggal penerimaan bantuan dikonversi ke format integer `YYYYMMDD` sehingga dapat digunakan sebagai `Tanggal_SK`. Prosedur juga menerapkan mekanisme anti-duplikasi menggunakan kondisi `NOT EXISTS`, sehingga hanya data `Penerima_ID` yang belum pernah dimuat ke fact yang akan dimasukkan. Dengan pendekatan ini, proses loading data ke fact dilakukan dengan konsisten, menjaga kualitas data, dan mencegah adanya baris ganda dalam Data Mart Kemahasiswaan.

3. Schedule ETL Jobs

Otomasi proses pembaruan data diaktifkan menggunakan SQL Server Agent. Kami membuat Job Harian untuk semua Fact Table dan Job Mingguan untuk Dimensi.

```
-- DEKLARASI VARIABEL SKALAR YANG DIGUNAKAN DALAM SCRIPT
DECLARE @NamaDatabase VARCHAR(100) = 'DM_Kemahasiswaan_DW';
DECLARE @ScheduleName VARCHAR(100) = 'Daily_Fact_Schedule';
DECLARE @NamaJob VARCHAR(100) = 'Execute_All_Fact_SP';
DECLARE @BikinJob BIT = 1; -- 0: Tidak Membuat/Mengupdate Job, 1:
Membuat/Mengupdate Job
DECLARE @ScheduleId INT;
DECLARE @JobId UNIQUEIDENTIFIER;

-- PASTIKAN NAMA DATABASE SAMA DENGAN DATABASE ANDA
SELECT @NamaDatabase = 'DM_Kemahasiswaan_DW';

-- 1. Membersihkan Objek Lama (Mencegah error "Job already exists")
IF EXISTS (SELECT 1 FROM msdb.dbo.sysjobs WHERE name = @NamaJob)
```

```

BEGIN
    -- Menghapus Schedule
    IF EXISTS (SELECT 1 FROM msdb.dbo.sysschedules WHERE name =
@ScheduleName)
    BEGIN
        EXEC msdb.dbo.sp_delete_schedule @schedule_name = @ScheduleName;
    END

    -- Menghapus Job
    IF EXISTS (SELECT 1 FROM msdb.dbo.sysjobs WHERE name = @NamaJob)
    BEGIN
        EXEC msdb.dbo.sp_delete_job @job_name = @NamaJob;
    END
END

-- 2. Membuat Job Utama
IF @BikinJob = 1
BEGIN
    EXEC msdb.dbo.sp_add_job
        @job_name = @NamaJob,
        @enabled = 1,
        @description = N'Job untuk memuat semua Fact Table (Partisipasi, Dana,
Prestasi, Beasiswa) secara harian.';

    -- 3. Mendapatkan Job ID
    SELECT @JobId = job_id FROM msdb.dbo.sysjobs WHERE name =
@NamaJob;

    -- 4. Membuat Job Step (Memanggil semua Stored Procedure)
    EXEC msdb.dbo.sp_add_jobstep
        @job_id = @JobId,
        @step_name = N'Execute_etl',
        @subsystem = N'TSQL',
        @command = N'EXEC dbo.SP_ETL_Fact_Partisipasi;
EXEC dbo.SP_ETL_Fact_Dana;
EXEC dbo.SP_ETL_Fact_Prestasi;
EXEC dbo.SP_ETL_Fact_Beasiswa;';
        @database_name = @NamaDatabase,
        @on_success_action = 3; -- Lanjut ke step berikutnya atau selesai

    -- 5. Membuat Schedule dan Melampirkannya ke Job
    EXEC msdb.dbo.sp_add_schedule
        @schedule_name = @ScheduleName,
        @enabled = 1,
        @freq_type = 4, -- Harian
        @freq_interval = 1, -- Setiap 1 hari
        @active_start_time = 000000, -- Mulai jam 00:00:00
        @schedule_id = @ScheduleId OUTPUT;

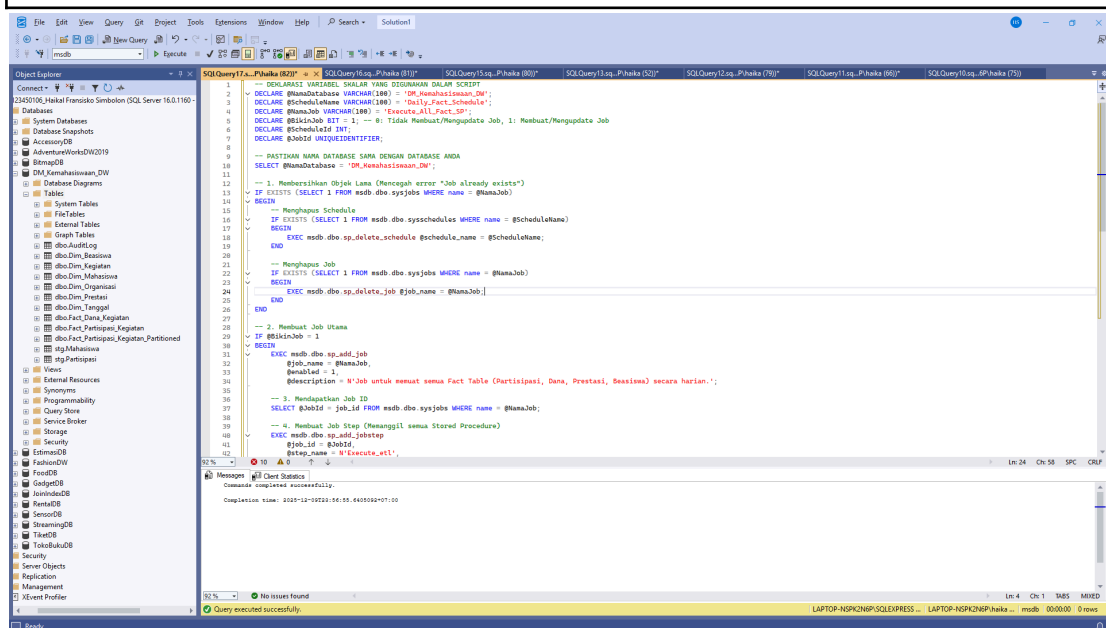
```

```
EXEC msdb.dbo.sp_attach_schedule
    @job_id = @JobId,
    @schedule_id = @ScheduleId;
```

-- 6. Set Job Response (Opsional: Memberi tahu jika gagal)

```
EXEC msdb.dbo.sp_update_job
    @job_id = @JobId,
    @notify_level_eventlog = 2,
    @notify_level_email = 2;

END
```



Script dimulai dengan deklarasi variabel-variabel penting seperti nama database (DM_Kemahasiswaan_DW), nama job (Execute_All_Fact_SP), dan nama schedule (Daily_Fact_Schedule). Bagian awal script melakukan pembersihan objek-objek lama dengan mengecek apakah job dan schedule dengan nama yang sama sudah ada sebelumnya, kemudian menghapusnya untuk mencegah error duplikasi. Ini adalah praktik yang baik dalam manajemen database untuk memastikan script dapat dijalankan berulang kali tanpa konflik.

Setelah pembersihan, script membuat job baru yang berisi satu step untuk menjalankan empat stored procedure secara berurutan: SP_ETL_Fact_Partisipasi, SP_ETL_Fact_Dana, SP_ETL_Fact_Prestasi, dan SP_ETL_Fact_Beasiswa. Job ini dijadwalkan untuk berjalan secara otomatis setiap hari pada pukul 00:00:00 (tengah malam) menggunakan SQL Server Agent Schedule. Dengan pengaturan @notify_level_eventlog = 2 dan @notify_level_email = 2, sistem akan mencatat dan mengirimkan notifikasi jika job gagal dieksekusi, memungkinkan administrator database untuk segera mengetahui dan menangani masalah pada proses ETL yang berjalan di data warehouse kemahasiswaan tersebut.

4.4 Dashboard Development

Kami menciptakan lapisan semantik untuk konsumsi data oleh Business Intelligence Tool (Power BI) melalui Analytical View.

1. Create Analytical Views

Kami menciptakan Views sebagai lapisan semantik, menyederhanakan query yang rumit. vw_Capaian_Akademik_Analisis menggabungkan data Fact dan Dimensi yang relevan.

```
USE DM_Kemahasiswaan_DW;
GO

-- View untuk Analisis Keterlibatan Mahasiswa dalam Kegiatan dan Organisasi
CREATE OR ALTER VIEW [vw_Keterlibatan_Mahasiswa_Analisis]
AS
SELECT
    -- Dimensi Mahasiswa (M)
    M.NIM,
    M>Nama_Mahasiswa,
    M.Fakultas,
    M.Program_Studi,
    M.Tahun_Masuk AS Angkatan,

    -- Data Partisipasi Kegiatan (FPK, DK, DT)
    DT.Tahun AS Tahun_Kegiatan,
    DT.FullDate AS Tanggal_Kegiatan,
    DK>Nama_Kegiatan,
    DK.Jenis_Kegiatan,
    DK.Skala_Kegiatan,
    FPK.Jumlah_Partisipan,

    -- Data Organisasi (DO)
    DO>Nama_Organisasi,
    DO.Jenis_Organisasi,
    DO.Status_Organisasi

FROM
    dbo.Dim_Mahasiswa M

-- LEFT JOIN ke Fact Partisipasi Kegiatan
-- Kita akan menggunakan Fact_Partisipasi_Kegiatan (tanpa Partisi)
LEFT JOIN dbo.Fact_Partisipasi_Kegiatan FPK
    ON M.Mahasiswa_SK = FPK.Mahasiswa_SK

-- LEFT JOIN ke Dimensi Kegiatan
LEFT JOIN dbo.Dim_Kegiatan DK
    ON FPK.Kegiatan_SK = DK.Kegiatan_SK

-- LEFT JOIN ke Dimensi Tanggal (untuk detail waktu kegiatan)
LEFT JOIN dbo.Dim_Tanggal DT
    ON FPK.Tanggal_SK = DT.Tanggal_SK
```



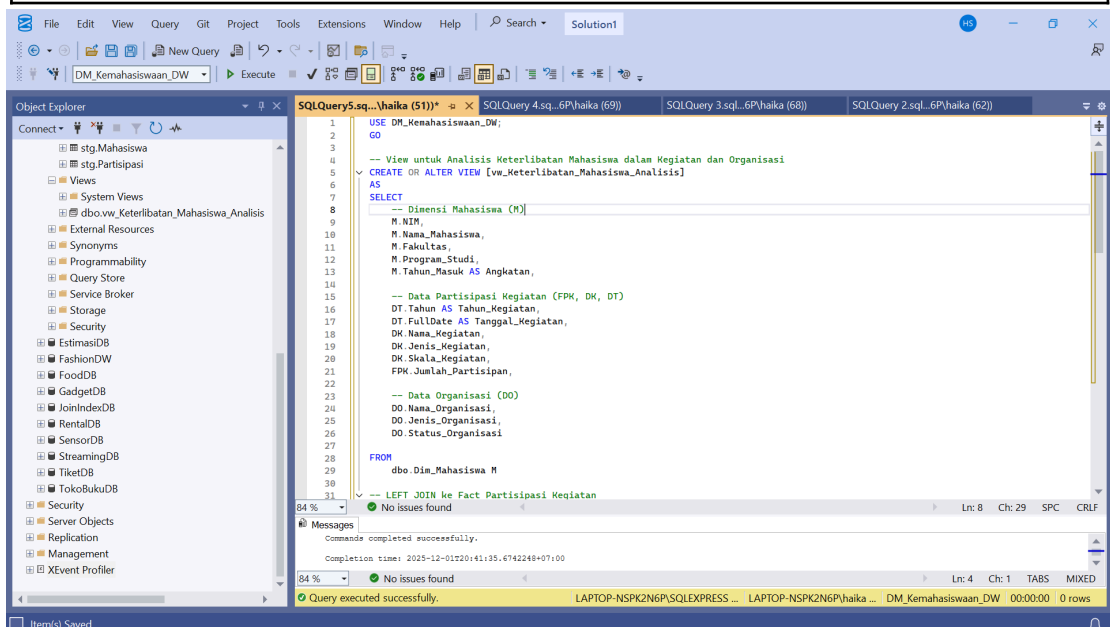
```

-- LEFT JOIN ke Dimensi Organisasi
LEFT JOIN dbo.Dim_Organisasi DO
  ON FPK.Organisasi_SK = DO.Organisasi_SK

-- Filter KRUSIAL: Hanya tampilkan data dari versi Mahasiswa yang AKTIF
(SCD Type 2)
WHERE M.IsCurrent = 1;
GO

-- Contoh kueri untuk menguji View yang sudah dibuat:
/*
SELECT TOP 100
  NIM,
  Nama_Mahasiswa,
  Fakultas,
  Tahun_Kegiatan,
  Nama_Kegiatan,
  Nama_Organisasi,
  Jumlah_Partisipan
FROM
  vw_Keterlibatan_Mahasiswa_Analisis
WHERE
  Nama_Kegiatan IS NOT NULL
ORDER BY
  Tahun_Kegiatan DESC;
*/

```



Bagian ini menjelaskan proses pengembangan dashboard dengan membuat lapisan semantik berupa Analytical Views agar data dapat dikonsumsi dengan mudah oleh Power BI. View vw_Keterlibatan_Mahasiswa_Analisis dibangun untuk menyederhanakan query kompleks dengan cara menggabungkan data dari fact dan

dimensi yang relevan. View ini mengambil informasi mahasiswa dari Dim_Mahasiswa, kemudian melakukan left join ke fact Fact_Partisipasi_Kegiatan dan berbagai dimensi terkait seperti Dim_Kegiatan, Dim_Tanggal, dan Dim_Organisasi. Struktur join ini menghasilkan satu tampilan analitis yang memadukan atribut mahasiswa, detail kegiatan, partisipasi, dan data organisasi dalam satu set data yang siap digunakan untuk visualisasi dashboard. Selain itu, filter WHERE M.IsCurrent = 1 memastikan hanya data mahasiswa aktif yang ditampilkan sesuai dengan prinsip SCD Type 2. Melalui pendekatan ini, dashboard dapat dibangun dengan lebih cepat karena query Power BI tinggal membaca view tersebut tanpa perlu merangkai join kompleks lagi, sehingga performa dan kemudahan analisis meningkat.

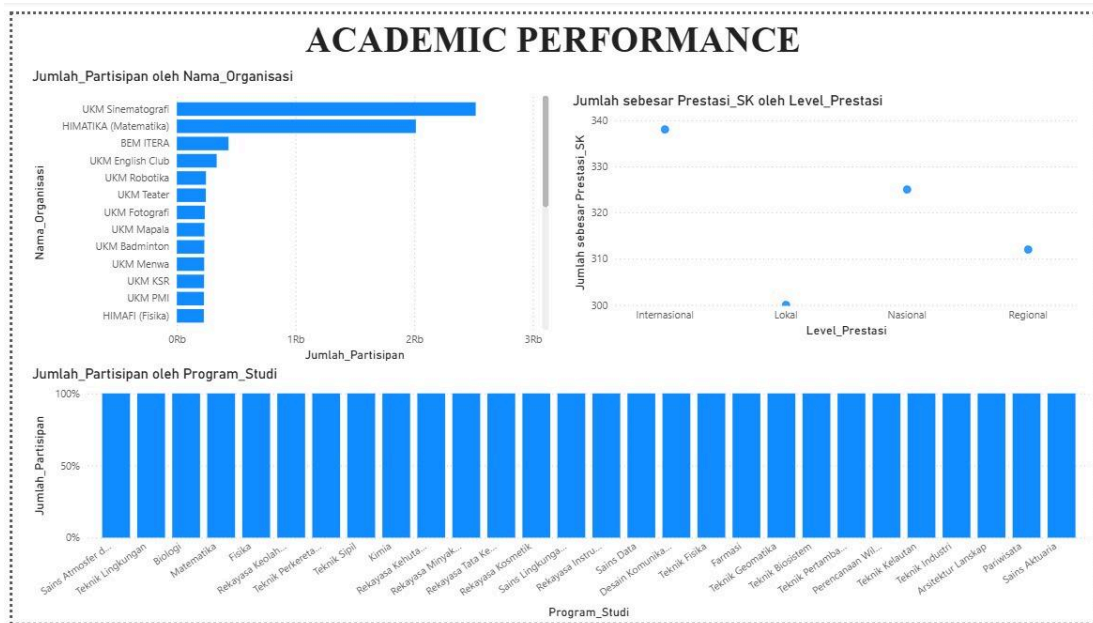
2. Design Power BI Dashboards

- Dashboard 1: Executive Summary



Dashboard Executive Summary menunjukkan bahwa mayoritas mahasiswa berada pada status aktif, sementara jumlah mahasiswa cuti dan drop out relatif kecil, yang mencerminkan kondisi akademik yang stabil dan mendukung tingginya potensi partisipasi kegiatan kemahasiswaan. Tren jumlah partisipan dan dana kegiatan per bulan memperlihatkan pola fluktuatif, dengan beberapa bulan mengalami lonjakan signifikan dan penurunan pada periode tertentu, menandakan bahwa pelaksanaan kegiatan serta penggunaan anggaran bersifat musiman dan dipengaruhi oleh agenda akademik maupun kalender kegiatan kampus. Dari sisi jenis kegiatan, partisipasi tertinggi berasal dari kegiatan lomba dan workshop, yang menunjukkan minat mahasiswa lebih besar pada aktivitas yang bersifat kompetitif dan pengembangan keterampilan dibandingkan kegiatan lainnya. Secara keseluruhan, dashboard ini memberikan gambaran ringkas mengenai keaktifan mahasiswa, dinamika partisipasi kegiatan, serta pola penggunaan dana, sehingga dapat digunakan sebagai dasar evaluasi dan pengambilan keputusan strategis oleh pimpinan institusi.

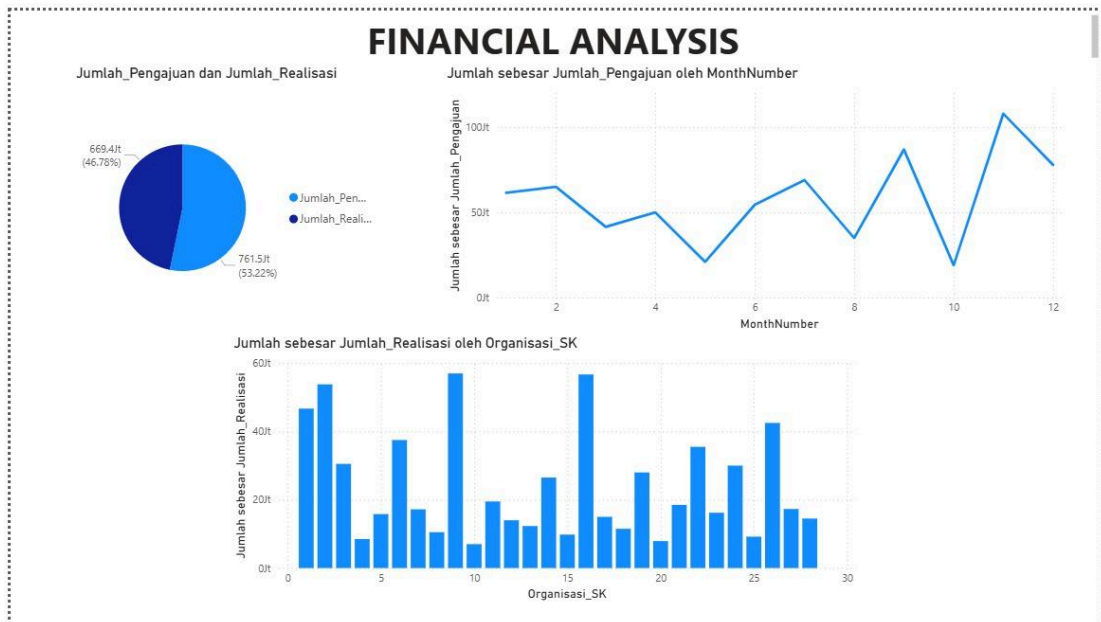
- Dashboard 2: Academic Performance



Dashboards "Academic Performance" ini menyajikan visualisasi data kinerja akademik mahasiswa dalam hal partisipasi dan prestasi pada berbagai kegiatan kemahasiswaan. Dashboard terdiri dari tiga panel visualisasi yang memberikan perspektif berbeda terhadap data partisipasi. Panel pertama di bagian kiri atas menampilkan bar chart horizontal yang menunjukkan jumlah partisipan berdasarkan nama organisasi, di mana UKM Sriwijaya memiliki jumlah partisipan tertinggi dengan sekitar 380 peserta, diikuti oleh HIMATIKA (Himpunan Mahasiswa Informatika) dan BEM ITERA dengan jumlah yang lebih rendah. Panel kedua di bagian kanan atas menampilkan scatter plot yang menggambarkan sebaran prestasi berdasarkan level (Internasional, Daerah, Nasional, dan Regional), di mana terlihat bahwa prestasi level Nasional memiliki jumlah sekitar 215, level Regional sekitar 213, level Daerah sekitar 335, dan level Internasional memiliki jumlah paling rendah.

Panel ketiga di bagian bawah dashboard menampilkan bar chart vertikal yang memvisualisasikan jumlah partisipan berdasarkan program studi. Grafik ini menunjukkan distribusi yang relatif merata di antara berbagai program studi dengan tingkat partisipasi konsisten sekitar 100% untuk setiap program. Program studi yang ditampilkan mencakup berbagai bidang seperti Teknik Sipil, Civil Engineering, Biologi, Matematika, Teknik Kimia, dan berbagai program lainnya yang tersebar secara horizontal. Visualisasi ini mengindikasikan bahwa tingkat keterlibatan mahasiswa dalam kegiatan akademik dan kemahasiswaan cukup seimbang di seluruh program studi yang ada. Dashboard ini secara keseluruhan memberikan gambaran komprehensif tentang partisipasi mahasiswa berdasarkan organisasi, tingkat prestasi yang dicapai, dan distribusi keterlibatan per program studi, yang berguna bagi manajemen institusi untuk mengevaluasi efektivitas program kemahasiswaan dan mengidentifikasi area yang memerlukan peningkatan atau dukungan lebih lanjut.

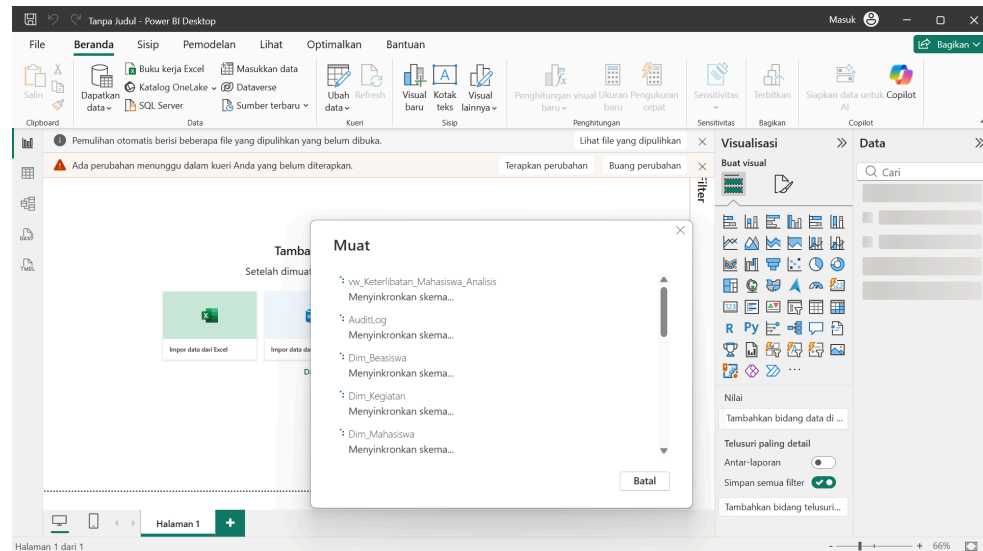
- Dashboard 3: Financial Analysis



Panel pertama di bagian kiri atas menampilkan pie chart yang membandingkan jumlah pengajuan dana dengan jumlah realisasi dana. Dari visualisasi tersebut terlihat bahwa jumlah pengajuan (ditandai dengan warna biru muda) sebesar 761.5K atau 53.52% lebih besar dibandingkan dengan jumlah realisasi (ditandai dengan warna biru tua) sebesar 661.4K atau 46.78%. Selisih ini mengindikasikan bahwa tidak semua dana yang diajukan berhasil direalisasikan, yang bisa disebabkan oleh berbagai faktor seperti penolakan proposal, pembatalan kegiatan, atau efisiensi penggunaan anggaran.

Panel kedua di bagian kanan atas menampilkan line chart yang memvisualisasikan tren jumlah pengajuan dana sepanjang tahun berdasarkan bulan (MonthNumber). Grafik menunjukkan fluktuasi yang cukup signifikan dengan pola naik-turun, di mana pengajuan tertinggi terjadi sekitar bulan ke-11 dengan nilai mencapai sekitar 100K, sementara titik terendah terjadi pada bulan ke-10. Pola ini menunjukkan adanya periode-periode tertentu di mana aktivitas pengajuan dana meningkat drastis, kemungkinan berkaitan dengan jadwal kegiatan akademik atau event besar di kampus. Panel ketiga di bagian bawah dashboard menampilkan bar chart vertikal yang menggambarkan jumlah realisasi dana berdasarkan organisasi kemahasiswaan (Organisasi_SK). Grafik menunjukkan distribusi yang bervariasi di antara sekitar 30 organisasi, dengan beberapa organisasi memiliki realisasi dana yang jauh lebih tinggi mencapai 60K, sementara organisasi lainnya memiliki realisasi yang lebih rendah berkisar antara 10K hingga 40K. Variasi ini mencerminkan perbedaan skala kegiatan, jumlah program yang dijalankan, atau prioritas pendanaan untuk masing-masing organisasi kemahasiswaan dalam institusi tersebut.

- Connect Power BI to SQL Server



Pada tahap ini menunjukkan bahwa tidak berhasil connect, sehingga kami hanya menggunakan data excel untuk membuat dashboardnya dan menggunakan Tableau

Step 3: Security Implementation

Implementasi keamanan mencakup pembatasan hak akses (prinsip Least Privilege), penyembunyian data sensitif (DDM), dan pelacakan aktivitas (Audit Trail).

1. Create Users and Assign Roles

Kami membuat akun khusus untuk Viewer (akses baca saja) dan Admin ETL (akses baca dan tulis).

```
USE DM_Kemahasiswaan_DW;
GO

-- 1. Membuat LOGIN & USER untuk Viewer BI (Contoh: Bakter/Stakeholder)

-- Menghapus LOGIN/USER lama jika sudah ada
IF EXISTS (SELECT name FROM sys.server_principals WHERE name =
'User_Bakti')
BEGIN
    -- Menghapus USER di database
    IF EXISTS (SELECT name FROM sys.database_principals WHERE name =
'User_Bakti' AND type_desc = 'SQL_USER')
        DROP USER User_Bakti;
    -- Menghapus LOGIN di Server
    DROP LOGIN User_Bakti;
END

-- Sekarang CREATE LOGIN dan USER (Dijamin tidak ada duplikasi)
CREATE LOGIN User_Bakti WITH PASSWORD='YourStrongPassword!!',
DEFAULT_DATABASE=[DM_Kemahasiswaan_DW];
CREATE USER User_Bakti FOR LOGIN User_Bakti;
-- Diberikan hak Read/Write (Untuk mengisi tabel Staging dan Fact)
```

```
ALTER ROLE [db_datareader] ADD MEMBER [User_Bakti];
ALTER ROLE [db_datawriter] ADD MEMBER [User_Bakti];
GO
```

-- 2. Membuat LOGIN & USER untuk ETL (Akses Staging & Fact)

```
-- Menghapus LOGIN/USER lama jika sudah ada
IF EXISTS (SELECT name FROM sys.server_principals WHERE name =
'User_ReadWrite')
BEGIN
    -- Menghapus USER di database
    IF EXISTS (SELECT name FROM sys.database_principals WHERE name =
'User_ReadWrite' AND type_desc = 'SQL_USER')
        DROP USER User_ReadWrite;
    -- Menghapus LOGIN di Server
    DROP LOGIN User_ReadWrite;
END
```

```
-- Sekarang CREATE LOGIN dan USER (Dijamin tidak ada duplikasi)
CREATE LOGIN User_ReadWrite WITH PASSWORD='YourStrongPassword!!',
DEFAULT_DATABASE=[DM_Kemahasiswaan_DW];
CREATE USER User_ReadWrite FOR LOGIN User_ReadWrite;
```

```
-- Diberikan hak Read/Write (Untuk mengisi tabel Staging dan Fact)
ALTER ROLE [db_datareader] ADD MEMBER [User_ReadWrite];
ALTER ROLE [db_datawriter] ADD MEMBER [User_ReadWrite];
GO
```

```
1 USE DM_Kemahasiswaan_DW;
2 GO
3
4 -- 1. Membuat LOGIN & USER untuk Viewer BI (Contoh: Bakter/Stakeholder)
5
6 -- Menghapus LOGIN/USER lama jika sudah ada
7 IF EXISTS (SELECT name FROM sys.server_principals WHERE name = 'User_Bakti')
8 BEGIN
9     -- Menghapus USER di database
10    IF EXISTS (SELECT name FROM sys.database_principals WHERE name = 'User_Bakti' AND type_desc = 'SQL_USER')
11        DROP USER User_Bakti;
12    -- Menghapus LOGIN di Server
13    DROP LOGIN User_Bakti;
14 END
15
16 -- Sekarang CREATE LOGIN dan USER (Dijamin tidak ada duplikasi)
17 CREATE LOGIN User_Bakti WITH PASSWORD='YourStrongPassword!!', DEFAULT_DATABASE=[DM_Kemahasiswaan_DW];
18 CREATE USER User_Bakti FOR LOGIN User_Bakti;
19 -- Diberikan hak Read/Write (Untuk mengisi tabel Staging dan Fact)
20 ALTER ROLE [db_datareader] ADD MEMBER [User_Bakti];
21 ALTER ROLE [db_datawriter] ADD MEMBER [User_Bakti];
22 GO
23
24 -- 2. Membuat LOGIN & USER untuk ETL (Akses Staging & Fact)
25
26 -- Menghapus LOGIN/USER lama jika sudah ada
27 IF EXISTS (SELECT name FROM sys.server_principals WHERE name = 'User_ReadWrite')
28 BEGIN
29     -- Menghapus USER di database
30    IF EXISTS (SELECT name FROM sys.database_principals WHERE name = 'User_ReadWrite' AND type_desc = 'SQL_USER')
31        DROP USER User_ReadWrite;
32    -- Menghapus LOGIN di Server
33    DROP LOGIN User_ReadWrite;
34 END
35
36 -- Sekarang CREATE LOGIN dan USER (Dijamin tidak ada duplikasi)
37 CREATE LOGIN User_ReadWrite WITH PASSWORD='YourStrongPassword!!', DEFAULT_DATABASE=[DM_Kemahasiswaan_DW];
38 CREATE USER User_ReadWrite FOR LOGIN User_ReadWrite;
39
40 Messages
41
42 Command completed successfully.
43 Completion time: 2020-10-10T00:00:01.000000000
```

Dimulai dengan proses pembuatan login dan user pertama bernama User_Bakti. Sebelum membuat user baru, script melakukan pengecekan terhadap keberadaan login dan user dengan

nama yang sama di level server dan database. Jika ditemukan, script akan menghapus user terlebih dahulu di level database, kemudian menghapus login di level server untuk menghindari konflik duplikasi. Setelah pembersihan selesai, script membuat login baru User_Bakti dengan password yang telah ditentukan dan menetapkan DM_Kemahasiswaan_DW sebagai database default. User ini kemudian ditambahkan ke dalam dua role: db_datareader yang memberikan hak untuk membaca data, dan db_datawriter yang memberikan hak untuk menulis atau memodifikasi data.

Selanjutnya script melakukan proses yang identik untuk user kedua bernama User_ReadWrite. Prosedur yang sama dijalankan: pengecekan keberadaan login dan user, penghapusan jika sudah ada, pembuatan login dan user baru dengan password yang sama, dan penetapan database default yang sama. User_ReadWrite juga diberikan role db_datareader dan db_datawriter, sehingga memiliki kemampuan akses yang setara dengan User_Bakti. Kedua user ini memiliki privilege untuk melakukan operasi baca (SELECT) dan tulis (INSERT, UPDATE, DELETE) terhadap semua tabel dalam database data warehouse kemahasiswaan, dengan mekanisme pembersihan otomatis yang memastikan script dapat dijalankan berulang kali tanpa menghasilkan error duplikasi.

2. Implement Data Masking

DDM diterapkan pada kolom identitas sensitif di Dim_Mahasiswa untuk menyamarkan data dari User_Rektor dan pengguna umum lainnya.

```
USE DM_Kemahasiswaan_DW;
GO

IF NOT EXISTS (SELECT 1 FROM sys.columns WHERE Name =
N'No_Telepon' AND Object_ID = Object_ID(N'dbo.Dim_Mahasiswa'))
BEGIN
    ALTER TABLE dbo.Dim_Mahasiswa
    ADD No_Telepon VARCHAR(15) NULL;
    PRINT 'Kolom No_Telepon berhasil ditambahkan.';
END
ELSE
    PRINT 'Kolom No_Telepon sudah ada, melanjutkan ke tahap masking.';
GO

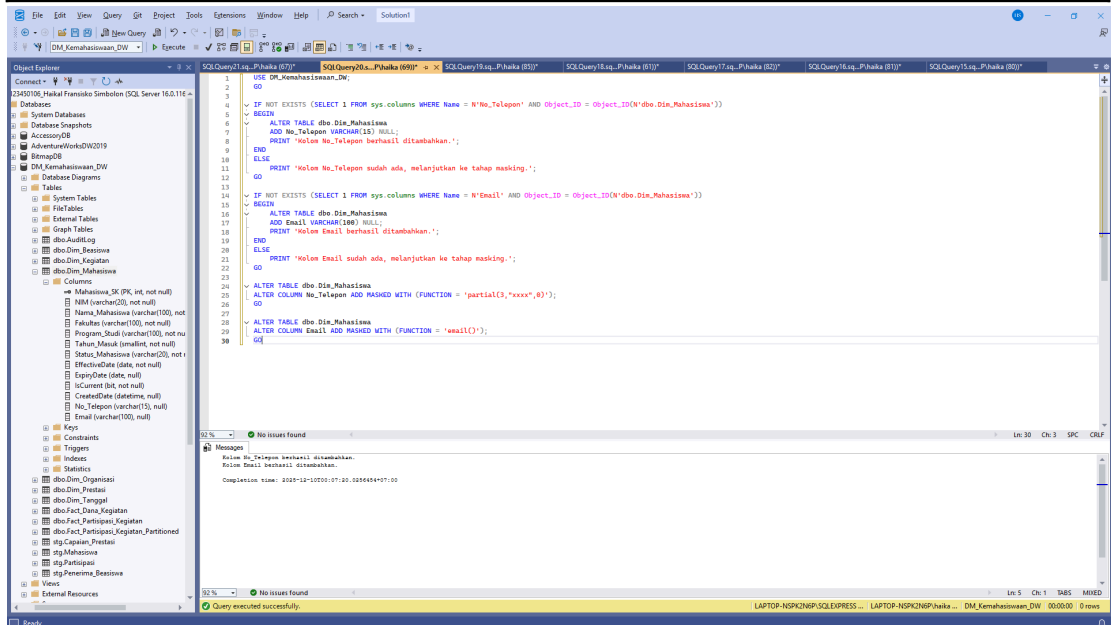
IF NOT EXISTS (SELECT 1 FROM sys.columns WHERE Name = N'Email' AND
Object_ID = Object_ID(N'dbo.Dim_Mahasiswa'))
BEGIN
    ALTER TABLE dbo.Dim_Mahasiswa
    ADD Email VARCHAR(100) NULL;
    PRINT 'Kolom Email berhasil ditambahkan.';
END
ELSE
    PRINT 'Kolom Email sudah ada, melanjutkan ke tahap masking.';
```


GO

```
ALTER TABLE dbo.Dim_Mahasiswa  
ALTER COLUMN No_Telepon ADD MASKED WITH (FUNCTION =  
'partial(3,"xxxx",0));
```

GO

```
ALTER TABLE dbo.Dim_Mahasiswa  
ALTER COLUMN Email ADD MASKED WITH (FUNCTION = 'email()');  
GO
```



Bagian ini menjelaskan implementasi Dynamic Data Masking (DDM) sebagai salah satu fitur keamanan untuk menyamarkan informasi sensitif pada tabel dimensi mahasiswa. Tujuan utamanya adalah melindungi privasi data seperti nomor telepon dan email, khususnya dari pengguna yang hanya memiliki hak akses baca, tetapi tidak memiliki kebutuhan untuk melihat data secara detail. Pada kolom No_Telepon, diterapkan fungsi masking `partial(3,"xxxx",0)` yang menampilkan tiga digit pertama secara jelas, sementara sisa digit diganti dengan teks "xxxx". Hal ini memungkinkan identifikasi dasar jika diperlukan, tanpa mengungkapkan informasi lengkap. Pada kolom Email, digunakan fungsi masking `email()` yang secara otomatis menghasilkan format email tersamarkan, misalnya `a***@m***.com`, sehingga struktur tetap terjaga tetapi identitas pengguna tidak terlihat. DDM bekerja secara dinamis di level query: data asli tetap tersimpan dalam bentuk lengkap, namun yang ditampilkan kepada pengguna yang tidak berhak hanyalah data hasil masking.

3. Implement Audit Trail

Kami mengimplementasikan SQL Server Audit untuk merekam aktivitas akses ke data sensitif, seperti akses SELECT ke Dim_Mahasiswa.

USE master;


```
GO
```

```
-- 1. CLEANUP DAN PENCEGAHAN ERROR DUPLIKASI
```

```
-- Hapus audit lama jika ada. Kami menggunakan satu nama yang konsisten:  
Audit_Akses_Data_Sensitif
```

```
IF EXISTS (SELECT * FROM sys.server_audits WHERE name = 'Audit_DW')  
BEGIN
```

```
    -- Matikan dulu sebelum dihapus
```

```
    ALTER SERVER AUDIT Audit_DW WITH (STATE = OFF);
```

```
    DROP SERVER AUDIT Audit_DW;
```

```
END
```

```
GO
```

```
-- 2. CREATE SERVER AUDIT (Pastikan PATH Valid)
```

```
CREATE SERVER AUDIT Audit_DW
```

```
TO FILE
```

```
(
```

```
    -- GANTI DENGAN PATH YANG BENAR-BENAR ADA DAN MEMILIKI  
IZIN TULIS!
```

```
    FILEPATH = 'D:\AUDIT DW'
```

```
    ,MAXSIZE = 50 MB
```

```
    ,MAX_ROLLOVER_FILES = 5
```

```
    ,RESERVE_DISK_SPACE = OFF
```

```
)
```

```
WITH
```

```
(
```

```
    QUEUE_DELAY = 1000,
```

```
    ON_FAILURE = CONTINUE
```

```
);
```

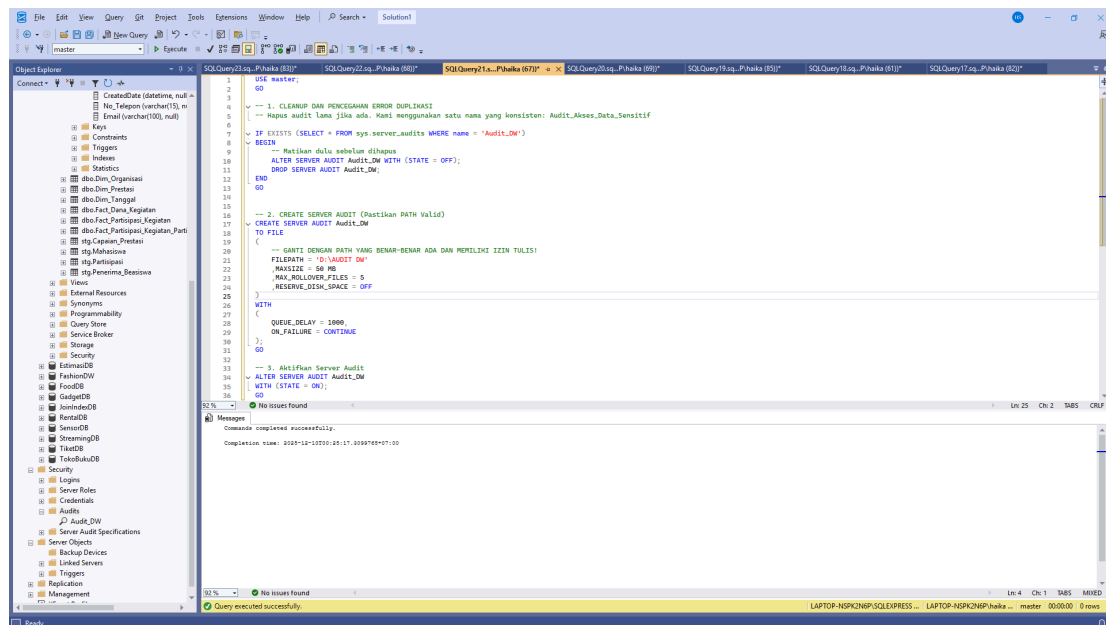
```
GO
```

```
-- 3. Aktifkan Server Audit
```

```
ALTER SERVER AUDIT Audit_DW
```

```
WITH (STATE = ON);
```

```
GO
```



Bagian pertama script berfokus pada pembersihan atau cleanup untuk mencegah error duplikasi. Script melakukan pengecekan terhadap keberadaan server audit dengan nama Audit_DW melalui system view sys.server_audits. Jika audit dengan nama tersebut sudah ada, sistem akan menonaktifkan audit terlebih dahulu menggunakan perintah ALTER SERVER AUDIT dengan parameter STATE = OFF, karena objek audit yang sedang dalam kondisi aktif tidak dapat langsung dihapus. Setelah berhasil dinonaktifkan, audit lama tersebut dihapus menggunakan perintah DROP SERVER AUDIT untuk memastikan tidak terjadi konflik saat pembuatan audit baru dengan nama yang sama.

Tahap berikutnya adalah pembuatan server audit baru bernama Audit_DW yang dikonfigurasi untuk menyimpan log aktivitas ke dalam file fisik. Lokasi penyimpanan file audit ditetapkan pada direktori D:\AUDIT DW dengan beberapa pengaturan teknis: ukuran maksimum setiap file dibatasi 50 MB, jumlah file rollover maksimal 5 file untuk sistem rotasi log otomatis, dan pengaturan RESERVE_DISK_SPACE = OFF yang berarti sistem tidak akan mereservasi ruang disk secara advance. Parameter QUEUE_DELAY diatur 1000 milidetik untuk mengontrol interval penulisan log ke file, sedangkan ON_FAILURE = CONTINUE memastikan SQL Server tetap beroperasi normal meskipun proses audit mengalami kegagalan. Setelah objek audit berhasil dibuat, script mengaktifkan audit tersebut menggunakan perintah ALTER SERVER AUDIT dengan STATE = ON, sehingga mekanisme audit mulai berjalan dan siap merekam event-event yang akan didefinisikan lebih lanjut melalui audit specification pada tahap berikutnya.

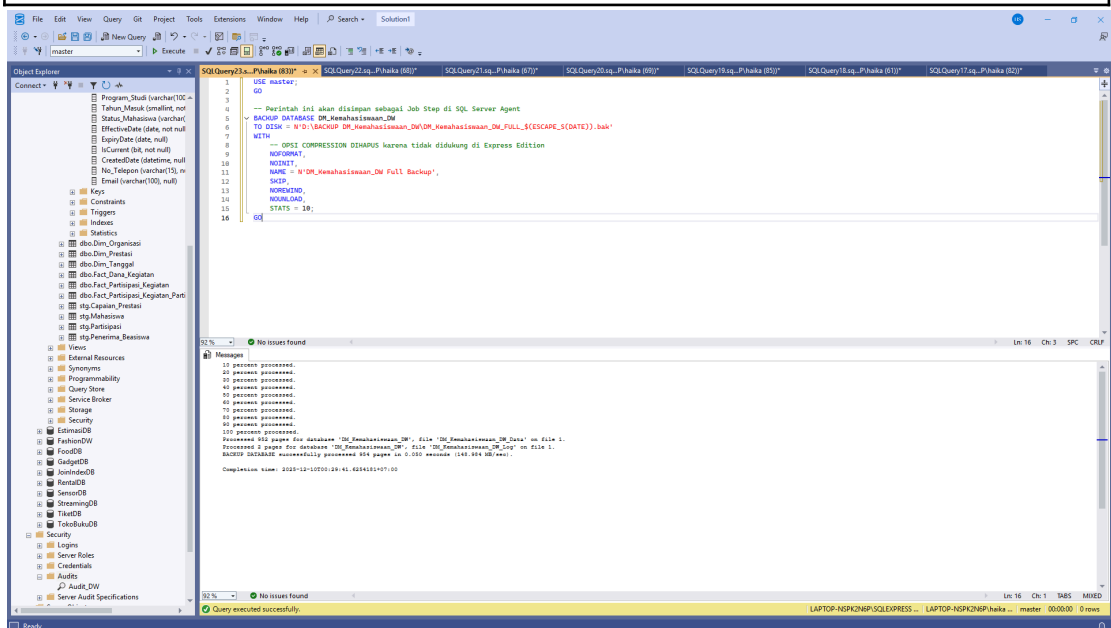
Step 4: Backup and Recovery Strategy

Karena Recovery Model sudah diatur ke FULL, kami dapat menerapkan strategi backup yang mendukung Point-in-Time Recovery. Untuk T-SQL for Backup Jobs. Kami membuat job step untuk Full Backup (Mingguan) dan Log Backup (Harian/Per Jam).

1. Full Database Backup (Mingguan)

```
USE master;  
GO
```

```
-- Perintah ini akan disimpan sebagai Job Step di SQL Server Agent  
BACKUP DATABASE DM_Kemahasiswaan_DW  
TO DISK = N'D:\BACKUP  
DM_Kemahasiswaan_DW\DM_Kemahasiswaan_DW_FULL_$(ESCAPE_S(DAT  
E)).bak'  
WITH  
    -- OPSI COMPRESSION DIHAPUS karena tidak didukung di Express Edition  
    NOFORMAT,  
    NOINIT,  
    NAME = N'DM_Kemahasiswaan_DW Full Backup',  
    SKIP,  
    NOREWIND,  
    NOUNLOAD,  
    STATS = 10;  
GO
```



Perintah BACKUP DATABASE digunakan untuk menyalin seluruh isi database ke dalam file fisik yang disimpan di lokasi D:\BACKUP DM_Kemahasiswaan_DW\ . Nama file backup menggunakan format dinamis dengan menyertakan variabel \$(ESCAPE_S(DATE)) yang akan menghasilkan timestamp tanggal saat backup dijalankan, sehingga setiap hasil backup tersimpan sebagai file terpisah dan tidak menimpa cadangan sebelumnya. Parameter NAME memberikan label deskriptif untuk backup ini sebagai 'DM_Kemahasiswaan_DW Full Backup' yang akan tercatat dalam metadata backup SQL Server, memudahkan identifikasi saat melakukan restore di kemudian hari.

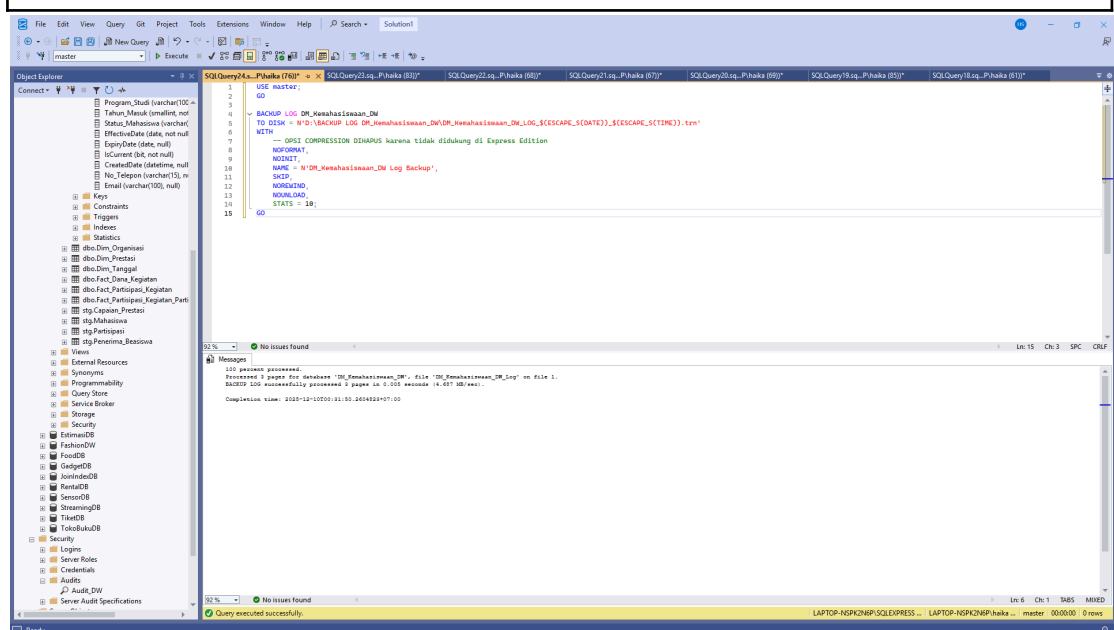
Konfigurasi backup dilengkapi dengan beberapa opsi teknis untuk memastikan proses berjalan sesuai kebutuhan. Parameter NOFORMAT mencegah media backup diformat ulang,

NOINIT memastikan backup ditambahkan ke media yang sudah ada tanpa menginisialisasi ulang, dan SKIP melewati pengecekan header backup untuk mempercepat proses. Opsi NOREWIND dan NOUNLOAD relevan untuk media tape namun tetap disertakan untuk konsistensi konfigurasi. Parameter STATS = 10 mengatur sistem untuk menampilkan progress backup setiap 10% penyelesaian, memberikan visibilitas real-time terhadap proses yang sedang berjalan. Penting dicatat bahwa opsi kompresi sengaja dihapus karena fitur tersebut tidak tersedia pada SQL Server Express Edition, sehingga file backup akan tersimpan dalam ukuran penuh tanpa kompresi untuk menjaga kompatibilitas dengan edisi yang digunakan.

2. Transaction Log Backup (Harian/Per Jam)

```
USE master;
GO

BACKUP LOG DM_Kemahasiswaan_DW
TO DISK = N'D:\BACKUP LOG
DM_Kemahasiswaan_DW\DM_Kemahasiswaan_DW_LOG_$(ESCAPE_S(
DATE
))_$(ESCAPE_S(TIME)).trn'
WITH
-- OPSI COMPRESSION DIHAPUS karena tidak didukung di Express Edition
NOFORMAT,
NOINIT,
NAME = N'DM_Kemahasiswaan_DW Log Backup',
SKIP,
NOREWIND,
NOUNLOAD,
STATS = 10;
GO
```



Kode ini menerapkan backup Transaction Log secara berkala untuk mendukung Point-in-Time Recovery. Perintah BACKUP LOG digunakan untuk mencadangkan hanya perubahan data yang terjadi sejak backup terakhir, sehingga ukuran file backup lebih kecil dan proses lebih cepat dibanding full backup. Hasil backup disimpan dalam folder D:\SQLBackup\Log dengan format nama file yang mengandung tanggal dan waktu ($\$(ESCAPE_S(\text{DATE}))_ \$(ESCAPE_S(\text{TIME}))$). Hal ini memastikan setiap file log tersimpan unik tanpa saling menimpa, sehingga administrator dapat memilih titik pemulihan yang tepat jika terjadi kegagalan. Penggunaan opsi COMPRESSION membantu mengurangi ukuran file, sedangkan parameter seperti NOFORMAT, NOINIT, dan SKIP memastikan media tidak di-reset dan tetap aman menyimpan backup sebelumnya. Opsi NOREWIND dan NOUNLOAD relevan untuk media tape namun tetap disertakan untuk konsistensi konfigurasi. Informasi kemajuan proses ditampilkan melalui STATS = 10, yang menunjukkan progres setiap 10%.

Step 5: User Acceptance Testing (UAT)

Sub-Step	Tujuan Utama	Jenis Aktivitas	Kebutuhan Kode
1. Create Test Cases	Mendefinisikan <i>skenario validasi</i> .	Dokumentasi dan perumusan kueri (T-SQL SELECT) untuk validasi data.	Skrip SQL SELECT untuk validasi data (DQ Checks).
2. Conduct UAT Sessions	Memvalidasi fungsionalitas dan tampilan <i>dashboard</i> oleh <i>end-users</i> (Stakeholders).	Pertemuan, <i>demonstrasi live</i> , dan pencatatan <i>feedback</i> .	Tidak ada kode baru.
3. Performance Testing	Mengukur <i>Query Response Time</i> dan <i>ETL Execution Time</i> .	Menjalankan kueri kompleks sambil mengaktifkan SET STATISTICS TIME ON dan SET STATISTICS IO ON.	Skrip SQL SELECT untuk <i>benchmarking</i> .
4. Refinement	Memperbaiki dan mengoptimalkan.	Implementasi perubahan <i>indexing</i> , <i>stored procedure</i> , atau <i>bug fix</i> .	Kode DDL/DML Modifikasi (Contoh: CREATE INDEX, ALTER PROCEDURE).

UAT adalah fase kritis untuk memastikan Data Warehouse (DW) dan Data Mart baru tidak hanya berfungsi secara teknis tetapi juga akurat, aman, dan memenuhi kebutuhan fungsional pengguna akhir (misalnya, Rektorat, Kepala Program Studi, dan Tim Keuangan).

1. Create Test Cases

Kami mengembangkan test case yang berfokus pada tiga area utama: Integritas Data, Fungsionalitas ETL, dan Keamanan Data.

Tujuan: Memastikan data yang baru dimuat ke Fact_Capaian_Prestasi dan Fact_Penerima_Beasiswa akurat, tidak duplikat, dan terhubung dengan benar ke Dimensi yang Aktif (IsCurrent = 1).

1. Data Quality Checks (DQ_001, DQ_004, DQ_005)

```
-- File: 08_Data_Quality_Checks_M3.sql
USE DM_Kemahasiswaan_DW;
GO

-----
-- DQ_001: Completeness - Cek NULL Foreign Keys di Fact_Partisipasi_Kegiatan
-- (Fact yang kolom FK-nya didefinisikan sebagai NOT NULL, kecuali
Organisasi_SK)
-----
PRINT '--- Checking DQ_001: NULL Foreign Keys in Fact_Partisipasi_Kegiatan
---';
-- Kolom SK yang NOT NULL di Fact_Partisipasi_Kegiatan: Tanggal_SK,
Mahasiswa_SK, Kegiatan_SK.
-- Organisasi_SK diizinkan NULL, tetapi tetap dicek jika ada kebutuhan bisnis
SELECT
    'Fact_Partisipasi_Kegiatan' AS TableName,
    COUNT(*) AS NullKeyCount,
    SUM(CASE WHEN Mahasiswa_SK IS NULL THEN 1 ELSE 0 END) AS
NullMahasiswa_SK, -- Mahasiswa_SK (NOT NULL)
    SUM(CASE WHEN Kegiatan_SK IS NULL THEN 1 ELSE 0 END) AS
NullKegiatan_SK, -- Kegiatan_SK (NOT NULL)
    SUM(CASE WHEN Tanggal_SK IS NULL THEN 1 ELSE 0 END) AS
NullTanggal_SK, -- Tanggal_SK (NOT NULL)
    SUM(CASE WHEN Organisasi_SK IS NULL THEN 1 ELSE 0 END) AS
NullOrganisasi_SK -- Organisasi_SK (NULLABLE)
FROM dbo.Fact_Partisipasi_Kegiatan
HAVING
    SUM(CASE WHEN Mahasiswa_SK IS NULL THEN 1 ELSE 0 END) > 0 OR
    SUM(CASE WHEN Kegiatan_SK IS NULL THEN 1 ELSE 0 END) > 0 OR
    SUM(CASE WHEN Tanggal_SK IS NULL THEN 1 ELSE 0 END) > 0 OR
    SUM(CASE WHEN Organisasi_SK IS NULL THEN 1 ELSE 0 END) > 0;
GO

-----
-- DQ_004: Consistency - Cek Fact yang Terhubung ke Record SCD Type 2 Lama
-- (Mengecek apakah ada data partisipasi yang masuk ke record Dim_Mahasiswa
lama)
-----
PRINT '--- Checking DQ_004: Fact Linked to Inactive SCD Records ---';
-- ASUMSI: Dim_Mahasiswa adalah Dimensi SCD Type 2 dengan kolom IsCurrent
```

```

SELECT
    'Fact_Partisipasi_Kegiatan' AS TableName,
    COUNT(f.Partisipasi_SK) AS InactiveSCDLinkCount
FROM dbo.Fact_Partisipasi_Kegiatan f
INNER JOIN dbo.Dim_Mahasiswa s ON f.Mahasiswa_SK = s.Mahasiswa_SK
WHERE s.IsCurrent = 0;
GO

```

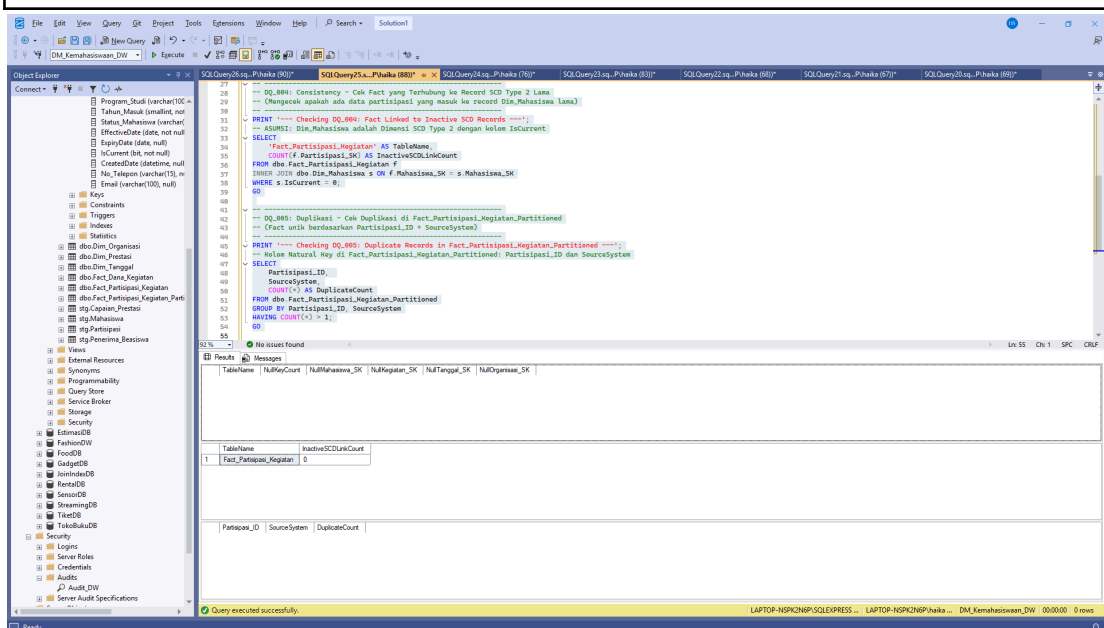
-- DQ_005: Duplikasi - Cek Duplikasi di Fact_Partisipasi_Kegiatan_Partitioned
 -- (Fact unik berdasarkan Partisipasi_ID + SourceSystem)

PRINT '--- Checking DQ_005: Duplicate Records in
 Fact_Partisipasi_Kegiatan_Partitioned ---';
 -- Kolom Natural Key di Fact_Partisipasi_Kegiatan_Partitioned: Partisipasi_ID dan
 SourceSystem

```

SELECT
    Partisipasi_ID,
    SourceSystem,
    COUNT(*) AS DuplicateCount
FROM dbo.Fact_Partisipasi_Kegiatan_Partitioned
GROUP BY Partisipasi_ID, SourceSystem
HAVING COUNT(*) > 1;
GO

```



Bagian ini bertujuan untuk menguji kualitas data yang telah dimuat ke dalam Data Mart. Serangkaian test case dibuat untuk memastikan bahwa data memenuhi tiga prinsip utama, yaitu: kelengkapan (completeness), konsistensi (consistency), dan tidak terjadi duplikasi (uniqueness). Seluruh kode dijalankan pada database DM_Kemahasiswaan_DW, sehingga seluruh pengecekan dilakukan terhadap tabel fact dan dimensi aktif yang menjadi bagian dari sistem data warehouse.

1. DQ_001 – Completeness: Pengecekan NULL Foreign Key

Query pertama berfungsi untuk mendeteksi apakah terdapat nilai NULL pada foreign key kritis di tabel Fact_Partisipasi_Kegiatan. Kolom seperti Mahasiswa_SK, Kegiatan_SK, dan Tanggal_SK didefinisikan sebagai NOT NULL, sehingga keberadaan nilai kosong akan menyebabkan integritas hubungan antar tabel gagal. Kolom Organisasi_SK diizinkan bernilai NULL berdasarkan aturan bisnis, pemeriksaan tetap dilakukan untuk keperluan dokumentasi. Query menghitung total baris serta jumlah NULL pada masing-masing kolom. Bagian HAVING digunakan untuk hanya menampilkan hasil jika terdapat minimal satu nilai NULL. Hal ini membantu tim memastikan bahwa setiap baris fact benar-benar memiliki referensi ke dimensi terkait, sehingga laporan analitik tidak mengandung data yang hilang atau rusak.

2. DQ_004 – Consistency: Deteksi Link ke Record SCD Lama

Pengecekan ketiga berfokus pada dimensi bertipe SCD Type 2, di mana perubahan informasi mahasiswa disimpan dalam versi historis. Query ini mengecek apakah fact table masih mengarah ke record dimensi yang sudah tidak aktif (IsCurrent = 0). Jika ditemukan, hal ini mengindikasikan bahwa proses ETL gagal memperbarui foreign key saat menjalankan dimensi terbaru. Kondisi ini dapat menyebabkan analisis berbasis status terkini menjadi tidak akurat. Oleh karena itu, hasil dari query ini digunakan untuk memverifikasi bahwa hubungan fact selalu menunjuk ke versi yang aktif.

3. DQ_005 – Uniqueness: Deteksi Duplikasi Data

Pengecekan terakhir bertujuan untuk mendeteksi duplikasi pada tabel Fact_Partisipasi_Kegiatan_Partitioned. Natural key dari fact ini adalah kombinasi Partisipasi_ID dan SourceSystem, sehingga kombinasi keduanya harus unik. Query melakukan GROUP BY berdasarkan kedua kolom tersebut dan menampilkan baris yang memiliki jumlah lebih dari satu (HAVING COUNT(*) > 1). Temuan duplikasi sangat penting karena dapat menyebabkan agregasi yang salah pada laporan dashboard, misalnya perhitungan jumlah partisipasi kegiatan menjadi berlipat. Jika ditemukan duplikat, tim dapat melakukan pembersihan data pada level staging atau memperbaiki logika ETL.

2. ETL Fungsionalitas (Simulasi ETL_002 - SCD Type 2)

```
USE DM_Kemahasiswaan_DW;  
GO
```

```
-- Mengubah prosedur untuk menggunakan nama kolom 'IsCurrent' yang benar
```

```
ALTER PROCEDURE dbo.usp_Load_Dim_Mahasiswa
```



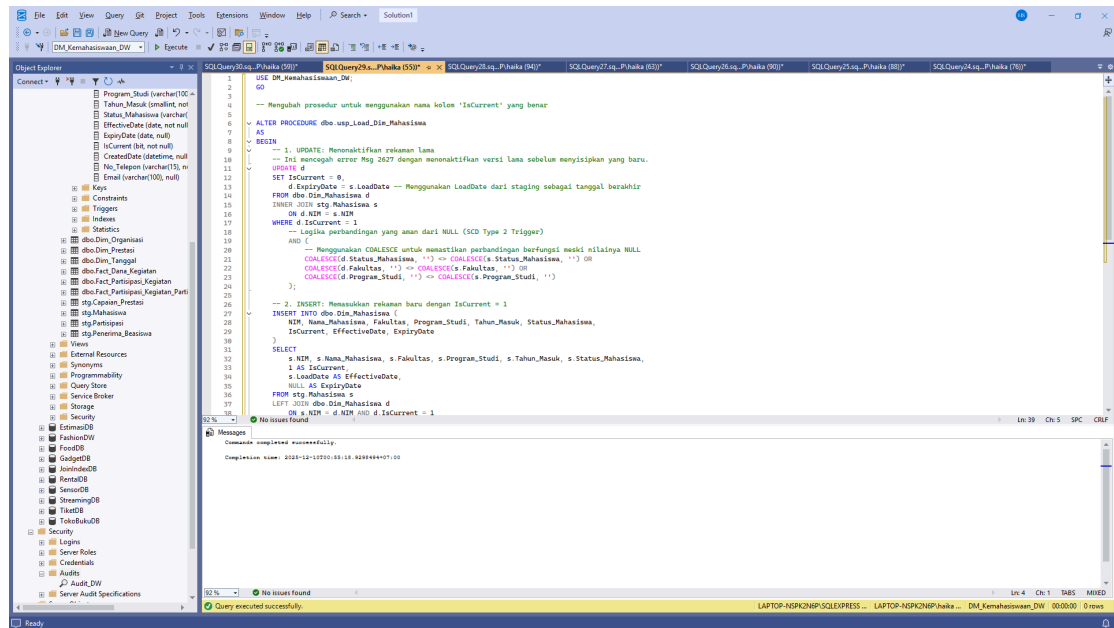
```

AS
BEGIN
    -- 1. UPDATE: Menonaktifkan rekaman lama
    -- Ini mencegah error Msg 2627 dengan menonaktifkan versi lama sebelum
    menyisipkan yang baru.
    UPDATE d
    SET IsCurrent = 0,
        d.ExpiryDate = s.LoadDate -- Menggunakan LoadDate dari staging sebagai
    tanggal berakhir
    FROM dbo.Dim_Mahasiswa d
    INNER JOIN stg.Mahasiswa s
        ON d.NIM = s.NIM
    WHERE d.IsCurrent = 1
        -- Logika perbandingan yang aman dari NULL (SCD Type 2 Trigger)
        AND (
            -- Menggunakan COALESCE untuk memastikan perbandingan berfungsi
    meski nilainya NULL
            COALESCE(d.Status_Mahasiswa, '') <> COALESCE(s.Status_Mahasiswa,
    '') OR
            COALESCE(d.Fakultas, '') <> COALESCE(s.Fakultas, '') OR
            COALESCE(d.Program_Studi, '') <> COALESCE(s.Program_Studi, '')
        );

    -- 2. INSERT: Memasukkan rekaman baru dengan IsCurrent = 1
    INSERT INTO dbo.Dim_Mahasiswa (
        NIM, Nama_Mahasiswa, Fakultas, Program_Studi, Tahun_Masuk,
    Status_Mahasiswa,
        IsCurrent, EffectiveDate, ExpiryDate
    )
    SELECT
        s.NIM, s>Nama_Mahasiswa, s.Fakultas, s.Program_Studi, s.Tahun_Masuk,
    s.Status_Mahasiswa,
        1 AS IsCurrent,
        s.LoadDate AS EffectiveDate,
        NULL AS ExpiryDate
    FROM stg.Mahasiswa s
    LEFT JOIN dbo.Dim_Mahasiswa d
        ON s.NIM = d.NIM AND d.IsCurrent = 1

    WHERE d.NIM IS NULL
        OR (d.NIM IS NOT NULL AND d.IsCurrent = 0);
END
GO

```



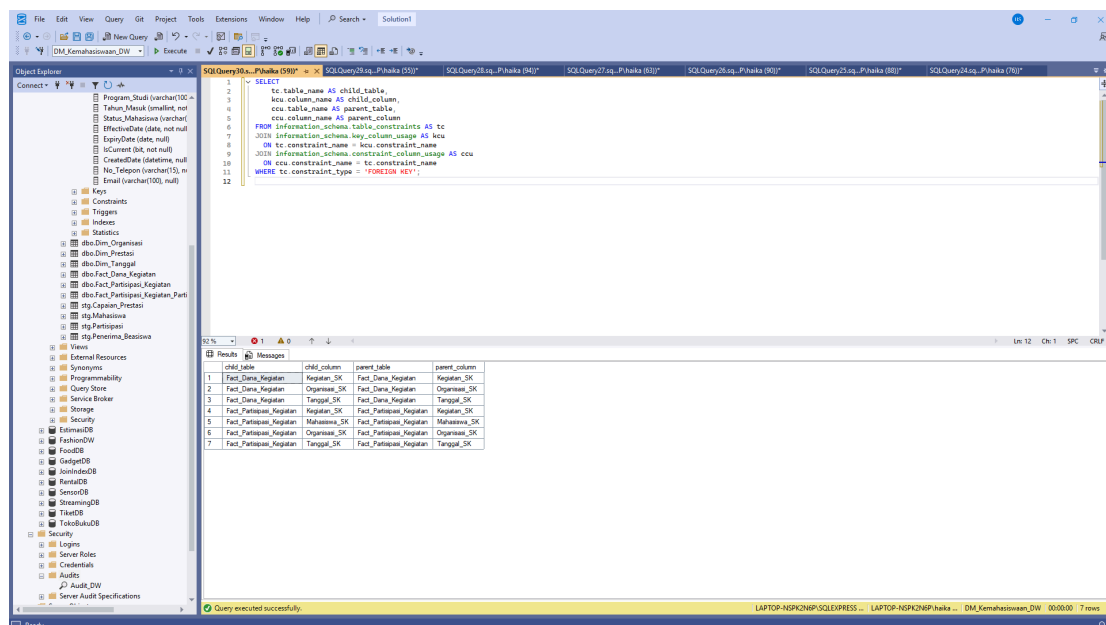
Modifikasi terhadap stored procedure `usp_Load_Dim_Mahasiswa` yang bertanggung jawab memuat data dimensi mahasiswa dengan menerapkan teknik Slowly Changing Dimension (SCD) Type 2. Stored procedure ini dirancang untuk menangani perubahan data historis pada dimensi mahasiswa dengan menyimpan versi lama dan versi baru dari record yang mengalami perubahan. Script menggunakan perintah `ALTER PROCEDURE` untuk mengubah prosedur yang sudah ada sebelumnya, memastikan nama kolom yang digunakan konsisten dengan struktur tabel, khususnya kolom `IsCurrent` yang menandai apakah suatu record merupakan versi terkini atau historis.

Proses pertama dalam stored procedure adalah operasi `UPDATE` yang menonaktifkan record lama ketika terdeteksi adanya perubahan pada atribut tertentu. Query melakukan join antara tabel dimensi `Dim_Mahasiswa` dengan tabel staging `stg.Mahasiswa` berdasarkan kolom `NIM`. Untuk setiap mahasiswa yang memiliki record aktif (`IsCurrent = 1`) dan mengalami perubahan pada kolom `Status_Mahasiswa`, `Fakultas`, atau `Program_Studi`, sistem akan mengubah flag `IsCurrent` menjadi 0 dan mengisi `ExpiryDate` dengan tanggal dari staging (`LoadDate`) untuk menandai kapan record tersebut tidak lagi berlaku. Fungsi `COALESCE` digunakan untuk menangani nilai `NULL` secara aman dalam perbandingan, memastikan bahwa perbandingan tetap berfungsi meskipun ada kolom yang bernilai `NULL`. Proses kedua adalah operasi `INSERT` yang memasukkan record baru atau versi terbaru dari data mahasiswa. Query mengambil data dari tabel staging dan melakukan `LEFT JOIN` dengan tabel dimensi untuk mengidentifikasi dua kondisi: mahasiswa baru yang belum ada di dimensi (`d.NIM IS NULL`) atau mahasiswa yang recordnya baru saja dinonaktifkan (`d.IsCurrent = 0`). Setiap record baru yang disisipkan akan memiliki `IsCurrent = 1` untuk menandai sebagai versi aktif, `EffectiveDate` diisi dengan `LoadDate` dari staging untuk mencatat kapan versi ini mulai berlaku, dan `ExpiryDate` diisi dengan `NULL` karena record ini masih berlaku hingga waktu yang belum ditentukan.

Step 6: Documentation

1. System Architecture Document

```
SELECT
    tc.table_name AS child_table,
    kc.column_name AS child_column,
    ccu.table_name AS parent_table,
    ccu.column_name AS parent_column
FROM information_schema.table_constraints AS tc
JOIN information_schema.key_column_usage AS kc
    ON tc.constraint_name = kc.constraint_name
JOIN information_schema.constraint_column_usage AS ccu
    ON ccu.constraint_name = tc.constraint_name
WHERE tc.constraint_type = 'FOREIGN KEY';
```



Script SQL ini melakukan query terhadap system view `information_schema` untuk mengidentifikasi seluruh relasi foreign key yang ada dalam database. Query menggabungkan tiga tabel metadata: `table_constraints` untuk mendapatkan informasi constraint, `key_column_usage` untuk mengidentifikasi kolom yang terlibat dalam constraint, dan `constraint_column_usage` untuk mengetahui tabel dan kolom referensi. Hasil query menampilkan empat kolom informasi: (`child_table`) yang memiliki foreign key, nama kolom foreign key di (`child_column`), nama tabel induk (`parent_table`) yang direferensikan, dan nama kolom primary key di tabel induk (`parent_column`). Filter `WHERE tc.constraint_type = 'FOREIGN KEY'` memastikan hanya constraint bertipe foreign key yang ditampilkan, menghasilkan dokumentasi lengkap tentang struktur relasional database yang berguna untuk memahami dependensi antar tabel dan integritas referensial dalam skema database.

2. Data Dictionary (Update dari Misi 1)

A. Fact Tables (Tabel Fakta)

Fact_Partisipasi_Kegiatan

Kolom	Tipe Data	PK/FK	Deskripsi	Busines Rule
Partisipasi_SK	INT	PK	Surrogate Key unik	Auto-increment, NOT NULL
Mahasiswa_SK	INT	FK	Key ke Dim_Mahasiswa	NOT NULL
Kegiatan_SK	INT	FK	Key ke Dim_Kegiatan	NOT NULL
Organisasi_SK	INT	FK	Key ke Dim_Organisasi	Dapat NULL
Tanggal_SK	INT	FK	Key ke Dim_Tanggal	NOT NULL
Jumlah_Partisipan	INT	Measure	Metrik partisipasi	Nilai selalu 1 (Additif)

Tabel Fact_Partisipasi_Kegiatan merupakan tabel fakta yang mencatat partisipasi mahasiswa dalam kegiatan dengan enam kolom utama. Partisipasi_SK sebagai Primary Key (INT) yang bersifat auto-increment dan NOT NULL untuk identifikasi unik setiap record. Tabel Fact_Partisipasi_Kegiatan memiliki empat Foreign Key yang menghubungkan ke dimensi terkait: Mahasiswa_SK ke Dim_Mahasiswa (NOT NULL), Kegiatan_SK ke Dim_Kegiatan (NOT NULL), Organisasi_SK ke Dim_Organisasi (dapat NULL karena tidak semua kegiatan diselenggarakan organisasi), dan Tanggal_SK ke Dim_Tanggal (NOT NULL). Kolom Jumlah_Partisipan (INT) merupakan measure yang selalu bernilai 1 dan bersifat aditif untuk menghitung total partisipasi. Struktur ini memungkinkan analisis partisipasi mahasiswa berdasarkan dimensi mahasiswa, kegiatan, organisasi, dan waktu secara multidimensional.

B. Fact_Capaian_Prestasi

Kolom	Tipe Data	PK/FK	Deskripsi	Business Rule
Prestasi_Fact_SK	INT	PK	Surrogate Key unik untuk setiap capaian	Auto-increment, NOT NULL
Mahasiswa_SK	INT	FK	Key ke Dim_Mahasiswa	NOT NULL
Prestasi_SK	INT	FK	Key ke Dim_Prestasi	NOT NULL
Tanggal_SK	INT	FK	Key ke Dim_Tanggal	NOT NULL

			(Tanggal capaian)	
Jumlah_Penghargaan	INT	Measure	Total penghargaan yang dicapai	Nilai selalu 1 (Additif)
Poin_Prestasi	DECIMAL (5,2)	Measure	Nilai bobot untuk Prestasi	Semi-Additif

Tabel Fact_Capaian_Prestasi merupakan tabel fakta yang mencatat prestasi mahasiswa dengan enam kolom. Prestasi_Fact_SK sebagai Primary Key (INT) yang auto-increment dan NOT NULL untuk identifikasi unik setiap capaian prestasi. Tabel ini memiliki tiga Foreign Key: Mahasiswa_SK ke Dim_Mahasiswa (NOT NULL), Prestasi_SK ke Dim_Prestasi (NOT NULL), dan Tanggal_SK ke Dim_Tanggal yang mencatat tanggal capaian prestasi (NOT NULL). Terdapat dua kolom measure: Jumlah_Penghargaan (INT) yang selalu bernilai 1 dan bersifat aditif untuk menghitung total penghargaan, serta Poin_Prestasi (DECIMAL 5,2) yang merekam nilai bobot prestasi dan bersifat semi-aditif karena poin tidak selalu dapat dijumlahkan langsung dalam semua konteks analisis. Struktur ini memungkinkan analisis prestasi mahasiswa berdasarkan dimensi mahasiswa, jenis prestasi, dan waktu pencapaian.

C. Fact_Dana_Kegiatan

Kolom	Tipe Data	PK/FK	Deskripsi	Business Rule
Dana_SK	INT	PK	Surrogate Key untuk transaksi dana	Auto-increment, NOT NULL
Kegiatan_SK	INT	FK	Key ke Dim_Kegiatan	NOT NULL
Tanggal_SK	INT	FK	Key ke Dim_Tanggal (Tanggal Realisasi)	NOT NULL
Jumlah_Pengajuan	INT	Measure	Total anggaran yang diajukan	Additif, Mata uang Rupiah
Jumlah_Realisasi	INT	Measure	Total dana yang direalisasikan	Additif, Digunakan untuk efisiensi

Tabel Fact_Dana_Kegiatan merupakan tabel fakta yang mencatat transaksi keuangan untuk setiap kegiatan dengan lima kolom. Dana_SK sebagai Primary Key (INT) yang auto-increment dan NOT NULL untuk identifikasi unik setiap transaksi dana. Tabel ini memiliki dua Foreign Key: Kegiatan_SK ke Dim_Kegiatan (NOT NULL) untuk

mengidentifikasi kegiatan yang didanai, dan Tanggal_SK ke Dim_Tanggal (NOT NULL) yang mencatat tanggal realisasi dana. Terdapat dua kolom measure yang keduanya bertipe INT dan bersifat aditif: Jumlah_Pengajuan yang merekam total anggaran yang diajukan dalam mata uang Rupiah, dan Jumlah_Realisasi yang mencatat total dana yang benar-benar direalisasikan, di mana perbandingan kedua measure ini digunakan untuk mengukur tingkat efisiensi penggunaan dana kegiatan. Struktur ini memungkinkan analisis keuangan kegiatan berdasarkan dimensi kegiatan dan waktu untuk evaluasi pengelolaan anggaran.

D. Fact_Penerima_Basiswa

Kolom	Tipe Data	PK/FK	Deskripsi	Business Rule
Beasiswa_Fact_SK	INT	PK	Surrogate Key untuk penerima beasiswa	Auto-increment, NOT NULL
Mahasiswa_SK	INT	FK	Key ke Dim_Mahasiswa	NOT NULL
Beasiswa_SK	INT	FK	Key ke Dim_Basiswa	NOT NULL
Tanggal_SK	INT	FK	Key ke Dim_Tanggal (Tanggal Mulai/Penerimaan)	NOT NULL
Jumlah_Penerima	INT	Measure	Indikator jumlah penerima beasiswa	Nilai selalu 1 (Additif)

Tabel Fact_Penerima_Basiswa merupakan tabel fakta yang mencatat data penerima beasiswa dengan lima kolom. Beasiswa_Fact_SK sebagai Primary Key (INT) yang auto-increment dan NOT NULL untuk identifikasi unik setiap record penerima beasiswa. Tabel ini memiliki tiga Foreign Key yang semuanya NOT NULL: Mahasiswa_SK ke Dim_Mahasiswa untuk mengidentifikasi mahasiswa penerima, Beasiswa_SK ke Dim_Basiswa untuk menentukan jenis beasiswa yang diterima, dan Tanggal_SK ke Dim_Tanggal yang mencatat tanggal mulai/penerimaan beasiswa. Kolom Jumlah_Penerima (INT) merupakan measure yang bersifat aditif dengan nilai selalu 1, berfungsi sebagai indikator untuk menghitung total jumlah penerima beasiswa dalam berbagai analisis dimensional. Struktur ini memungkinkan analisis distribusi beasiswa berdasarkan dimensi mahasiswa, jenis beasiswa, dan periode waktu untuk evaluasi program bantuan mahasiswa.

3. ETL Documentation

- ETL Package

```
USE DM_Kemahasiswaan_DW;
GO

/* 0. CREATE SCHEMA ETL (PACKAGE ROOT) */
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name = 'etl')
    EXEC('CREATE SCHEMA etl');
GO

/* 1. CREATE ETL LOG TABLE */
IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE name = 'ETL_Log')
BEGIN
    CREATE TABLE etl.ETL_Log (
        LogID INT IDENTITY(1,1) PRIMARY KEY,
        ETL_Procedure VARCHAR(200),
        Status VARCHAR(20),
        ErrorMessage VARCHAR(MAX) NULL,
        LogDate DATETIME DEFAULT GETDATE()
    );
END;
GO

/* 2. PROCEDURE – LOAD DIM PROGRAM (SCD Type 1) */

IF EXISTS (SELECT 1 FROM sys.objects WHERE name = 'Load_DimProgram')
    DROP PROCEDURE etl.Load_DimProgram;
GO

CREATE PROCEDURE etl.Load_DimProgram
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @ProcName VARCHAR(200) = 'Load_DimProgram';

    BEGIN TRY
        BEGIN TRANSACTION;

        -- Extract & Transform (Asumsi stg.Program sudah ada)
        SELECT
            ProgramCode,
            UPPER(LTRIM(RTRIM(ProgramName))) AS ProgramName,
            UPPER(LTRIM(RTRIM(Faculty))) AS Faculty
        INTO #Temp_Program
        FROM stg.Program
        WHERE ProgramCode IS NOT NULL;

        -- Load (MERGE untuk INSERT dan UPDATE - SCD Type 1)
```

```

MERGE INTO dbo.Dim_Program AS T
USING #Temp_Program AS S
  ON T.ProgramCode = S.ProgramCode
  WHEN MATCHED AND (
    T.ProgramName <> S.ProgramName OR T.Faculty <> S.Faculty -- Cek
    perubahan
  ) THEN
    UPDATE SET
      T.ProgramName = S.ProgramName,
      T.Faculty = S.Faculty
  WHEN NOT MATCHED BY TARGET THEN
    INSERT (ProgramCode, ProgramName, Faculty)
    VALUES (S.ProgramCode, S.ProgramName, S.Faculty);

DROP TABLE #Temp_Program;

INSERT INTO etl.ETL_Log (ETL_Procedure, Status)
VALUES (@ProcName, 'SUCCESS');

COMMIT TRANSACTION;
END TRY
BEGIN CATCH
  IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;

  INSERT INTO etl.ETL_Log (ETL_Procedure, Status, ErrorMessage)
  VALUES (@ProcName, 'FAILED', ERROR_MESSAGE());
END CATCH;
END;
GO

/* 3. PROCEDURE – LOAD DIM STUDENT (SCD Type 2) */
IF EXISTS (SELECT 1 FROM sys.objects WHERE name = 'Load_DimStudent')
  DROP PROCEDURE etl.Load_DimStudent;
GO

CREATE PROCEDURE etl.Load_DimStudent
AS
BEGIN
  SET NOCOUNT ON;
  DECLARE @ProcName VARCHAR(200) = 'Load_DimStudent';

  -- Kolom yang memicu SCD Type 2: StudentName, ProgramCode, Status
  BEGIN TRY
    BEGIN TRANSACTION;

    -- 1. Expire old records (Jika ada perubahan pada kolom pemicu)
    UPDATE T
    SET
      T.ExpiryDate = GETDATE(),

```



```

        T.IsCurrent = 0
    FROM dbo.Dim_Student T
    INNER JOIN stg.Student S ON T.NIM = S.NIM
    WHERE T.IsCurrent = 1
    AND (
        T.StudentName <> S.StudentName
        OR T.ProgramCode <> S.ProgramCode
        OR T.Status <> S.Status
    );

-- 2. Insert new/updated records (Termasuk data baru dan data yang berubah)
INSERT INTO dbo.Dim_Student (
    NIM, StudentName, Gender, BirthDate, EnrollmentDate, ProgramCode,
    ProgramName, Faculty, EntryYear, Status, EffectiveDate, IsCurrent
)
SELECT
    S.NIM,
    UPPER(TRIM(S.StudentName)),
    S.Gender, S.BirthDate, S.EnrollmentDate,
    S.ProgramCode,
    P.ProgramName, -- Lookup dari Dim_Program
    P.Faculty, -- Lookup dari Dim_Program
    YEAR(S.EnrollmentDate),
    S.Status,
    GETDATE(), -- EffectiveDate
    1 -- IsCurrent
FROM stg.Student S
LEFT JOIN dbo.Dim_Program P ON S.ProgramCode = P.ProgramCode
WHERE NOT EXISTS (
    SELECT 1
    FROM dbo.Dim_Student D
    WHERE D.NIM = S.NIM AND D.IsCurrent = 1
);

INSERT INTO etl.ETL_Log (ETL_Procedure, Status)
VALUES (@ProcName, 'SUCCESS');

COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;

    INSERT INTO etl.ETL_Log (ETL_Procedure, Status, ErrorMessage)
    VALUES (@ProcName, 'FAILED', ERROR_MESSAGE());
END CATCH;
END;
GO

```

```

/* 4. MASTER PROCEDURE – MENJALANKAN SEMUA */
IF EXISTS (SELECT 1 FROM sys.objects WHERE name =
'Master_ETL_Mahasiswa')
    DROP PROCEDURE etl.Master_ETL_Mahasiswa;
GO

CREATE PROCEDURE etl.Master_ETL_Mahasiswa
AS
BEGIN
    PRINT 'Memulai ETL Data Mart Kemahasiswaan...';

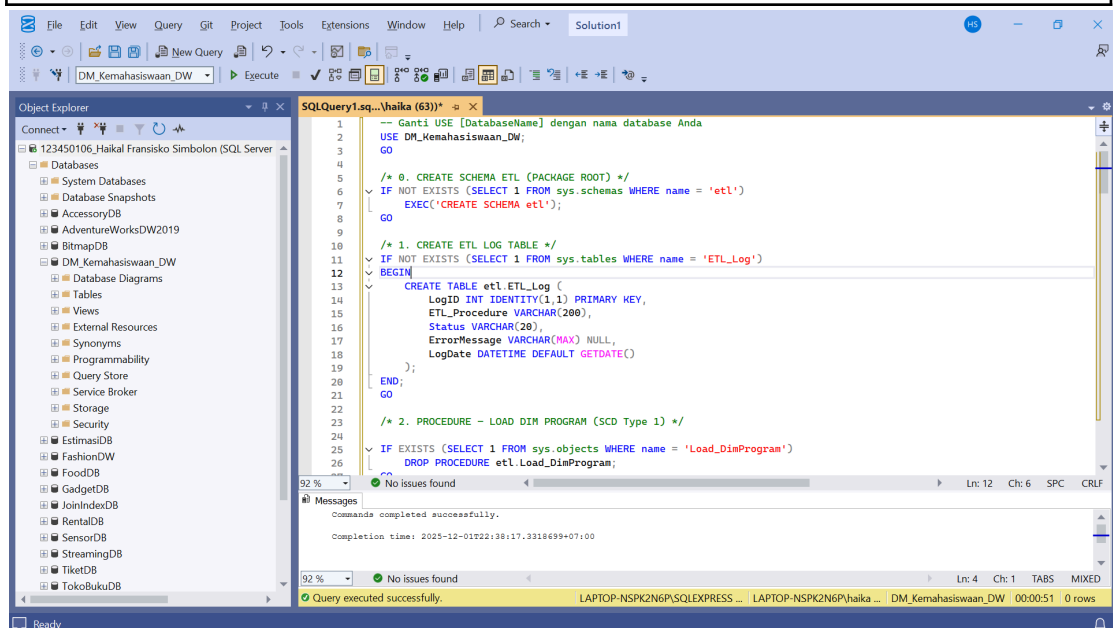
    -- Step 1: Load Dimensions
    EXEC etl.Load_DimProgram;
    EXEC etl.Load_DimStudent;

    -- TO DO: Tambahkan prosedur untuk Load Fact Tables (e.g.,
    Load_FactActivityParticipation)

    PRINT 'ETL Dimensi Kemahasiswaan selesai.';
END;
GO

/* END OF PACKAGE */

```



Paket ETL Documentation membangun proses pemuatan data yang terstruktur dengan membuat schema etl, tabel log, serta rangkaian prosedur untuk memproses dimensi. Schema etl berfungsi sebagai wadah terpusat, sementara tabel etl.ETL_Log digunakan untuk mencatat status dan pesan kesalahan setiap eksekusi sebagai audit trail. Prosedur Load_DimProgram menerapkan SCD Type 1 menggunakan perintah MERGE, sehingga perubahan atribut seperti nama program atau fakultas langsung diperbarui tanpa menyimpan versi lama, dan seluruh proses dibungkus dalam

transaksi untuk menjaga konsistensi. Sebaliknya, prosedur Load_DimStudent mengimplementasikan SCD Type 2, yaitu mendeteksi perubahan pada atribut mahasiswa, menandai record lama sebagai tidak aktif (IsCurrent = 0) dan memasukkan record baru dengan EffectiveDate sebagai riwayat. Lookup ke Dim_Program memastikan data fakultas dan program tetap sinkron. Semua keberhasilan dan kegagalan dicatat ke tabel log. Prosedur terakhir, Master_ETL_Mahasiswa, bertindak sebagai orkestrator untuk menjalankan seluruh prosedur ETL secara berurutan dan siap diperluas untuk memuat tabel fact, sehingga keseluruhan paket ETL menjadi modular, terdokumentasi, dan siap dijalankan otomatis melalui SQL Server Agent.

- ETL Script

```
USE DM_Kemahasiswaan_DW;
GO

/* 0. SETUP AWAL: Membuat Log Table (dbo.ETL_Log) */
-----

IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE name =
'ETL_Log')
BEGIN
    CREATE TABLE dbo.ETL_Log (
        LogID INT IDENTITY(1,1) PRIMARY KEY,
        ETL_Procedure VARCHAR(200),
        Status VARCHAR(20),
        ErrorMessage VARCHAR(MAX) NULL,
        LogDate DATETIME DEFAULT GETDATE()
    );
END;
GO

/* 1. ETL DIM_PROGRAM (SCD Type 1: Update jika Nama/Fakultas
berubah) */
-----

BEGIN TRY
    PRINT 'Memulai ETL DIM_PROGRAM (SCD Type 1)...';
    BEGIN TRANSACTION;

    -- ** PERHATIAN: GANTI 'stg.Program' jika nama tabel staging Anda
berbeda **
    SELECT ProgramCode, ProgramName, Faculty
    INTO #Temp_Program
    FROM stg.Program
    WHERE ProgramCode IS NOT NULL;

    -- Load (MERGE)
```

```

MERGE INTO dbo.Dim_Program AS Target
USING #Temp_Program AS Source
  ON Target.ProgramCode = Source.ProgramCode
WHEN MATCHED AND (
  -- Perbandingan NULL-safe untuk pemicu SCD 1
  COALESCE(Target.ProgramName, "") <>
COALESCE(Source.ProgramName, "") OR
  COALESCE(Target.Faculty, "") <> COALESCE(Source.Faculty, "")
) THEN
  UPDATE SET
    Target.ProgramName = Source.ProgramName,
    Target.Faculty = Source.Faculty
WHEN NOT MATCHED THEN
  INSERT (ProgramCode, ProgramName, Faculty)
  VALUES (Source.ProgramCode, Source.ProgramName,
Source.Faculty);

DROP TABLE #Temp_Program;

INSERT INTO dbo.ETL_Log (ETL_Procedure, Status)
VALUES ('Load_DimProgram', 'SUCCESS');

COMMIT TRANSACTION;
END TRY
BEGIN CATCH
  IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
  INSERT INTO dbo.ETL_Log (ETL_Procedure, Status, ErrorMessage)
  VALUES ('Load_DimProgram', 'FAILED', ERROR_MESSAGE());
END CATCH;
GO

/* 2. ETL DIM_STUDENT (SCD Type 2: Expire & Insert jika Status/Prodi
berubah) */
-----

BEGIN TRY
  PRINT 'Memulai ETL DIM_STUDENT (SCD Type 2)...';
  BEGIN TRANSACTION;

  -- ** PERHATIAN: GANTI NAMA TABEL/KOLOM STAGING jika
berbeda **
  SELECT NIM, Nama_Mahasiswa, Gender, EnrollmentDate,
ProgramCode, Status_Mahasiswa
  INTO #Temp_Student
  FROM stg.Student
  WHERE NIM IS NOT NULL;

```

-- 2A. EXPIRY (UPDATE): Menonaktifkan rekaman lama yang memiliki perubahan

```
UPDATE Target
SET
    Target.ExpiryDate = Source.LoadDate,
    Target.IsCurrent = 0
FROM dbo.Dim_Mahasiswa AS Target
INNER JOIN #Temp_Student AS Source ON Target.NIM = Source.NIM
WHERE Target.IsCurrent = 1
    AND (
        COALESCE(Target>Nama_Mahasiswa, ") <>
COALESCE(Source>Nama_Mahasiswa, ") OR
        COALESCE(Target.ProgramCode, ") <>
COALESCE(Source.ProgramCode, ") OR
        COALESCE(Target.Status_Mahasiswa, ") <>
COALESCE(Source.Status_Mahasiswa, ")
    );
```

-- 2B. INSERT: Load New/Changed Records

```
INSERT INTO dbo.Dim_Mahasiswa ( -- ASUMSI: Target adalah
dbo.Dim_Mahasiswa
    NIM, Nama_Mahasiswa, Gender, EnrollmentDate, ProgramCode,
    ProgramName, Faculty, EntryYear, Status_Mahasiswa, EffectiveDate,
    IsCurrent
)
SELECT
    S.NIM,
    UPPER(TRIM(S>Nama_Mahasiswa)),
    S.Gender,
    S.EnrollmentDate,
    S.ProgramCode,
    P.ProgramName, -- Lookup dari Dim_Program
    P.Faculty,    -- Lookup dari Dim_Program
    YEAR(S.EnrollmentDate),
    S.Status_Mahasiswa,
    GETDATE() AS EffectiveDate,
    1 AS IsCurrent
FROM #Temp_Student S
LEFT JOIN dbo.Dim_Program P ON S.ProgramCode = P.ProgramCode
WHERE NOT EXISTS (
    -- Hanya masukkan jika TIDAK ADA record aktif (IsCurrent=1) untuk
    NIM ini.
    SELECT 1
    FROM dbo.Dim_Mahasiswa D
    WHERE D.NIM = S.NIM AND D.IsCurrent = 1
);

DROP TABLE #Temp_Student;
```

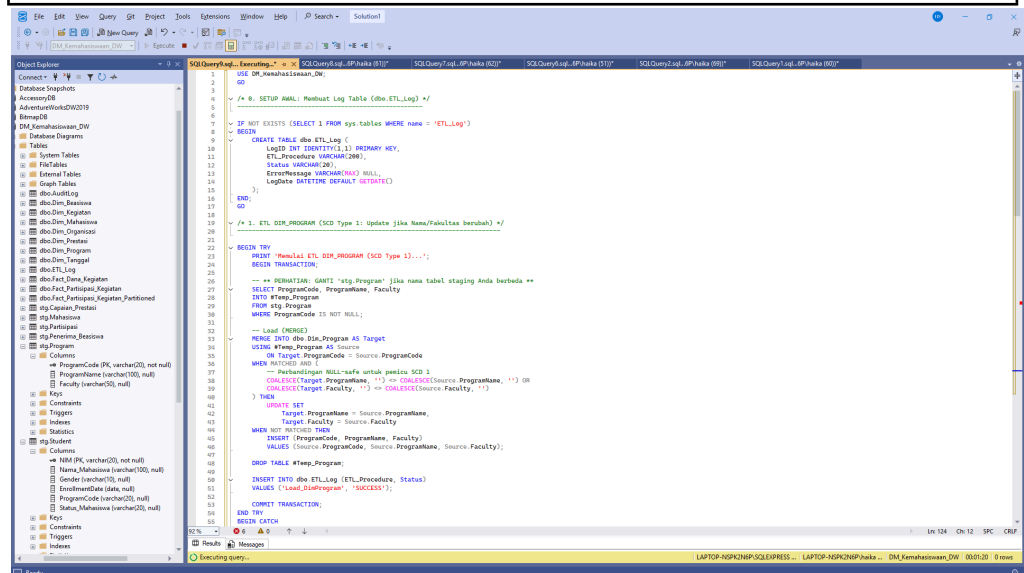
```

INSERT INTO dbo.ETL_Log (ETL_Procedure, Status)
VALUES ('Load_DimStudent', 'SUCCESS');

COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    INSERT INTO dbo.ETL_Log (ETL_Procedure, Status, ErrorMessage)
    VALUES ('Load_DimStudent', 'FAILED', ERROR_MESSAGE());
END CATCH;
GO

/* ETL SELESAI */
PRINT 'ETL Dimensi Kemahasiswaan Selesai.';
GO

```



Proses Extract, Transform, Load (ETL) untuk memuat data dimensi pada database data warehouse kemahasiswaan (DM_Kemahasiswaan_DW). Bagian awal script membuat tabel log bernama ETL_Log jika belum ada, yang berfungsi untuk mencatat setiap eksekusi prosedur ETL beserta statusnya (SUCCESS atau FAILED) dan pesan error jika terjadi kegagalan. Tabel log ini memiliki kolom LogID sebagai primary key auto-increment, ETL_Procedure untuk nama prosedur yang dijalankan, Status untuk status eksekusi, ErrorMessage untuk menyimpan detail error, dan LogDate yang secara otomatis mencatat waktu kejadian. Setiap proses ETL dibungkus dalam blok TRY-CATCH dengan mekanisme transaksi untuk memastikan data integrity, di mana jika terjadi error maka seluruh operasi akan di-rollback dan detail error akan dicatat ke tabel log.

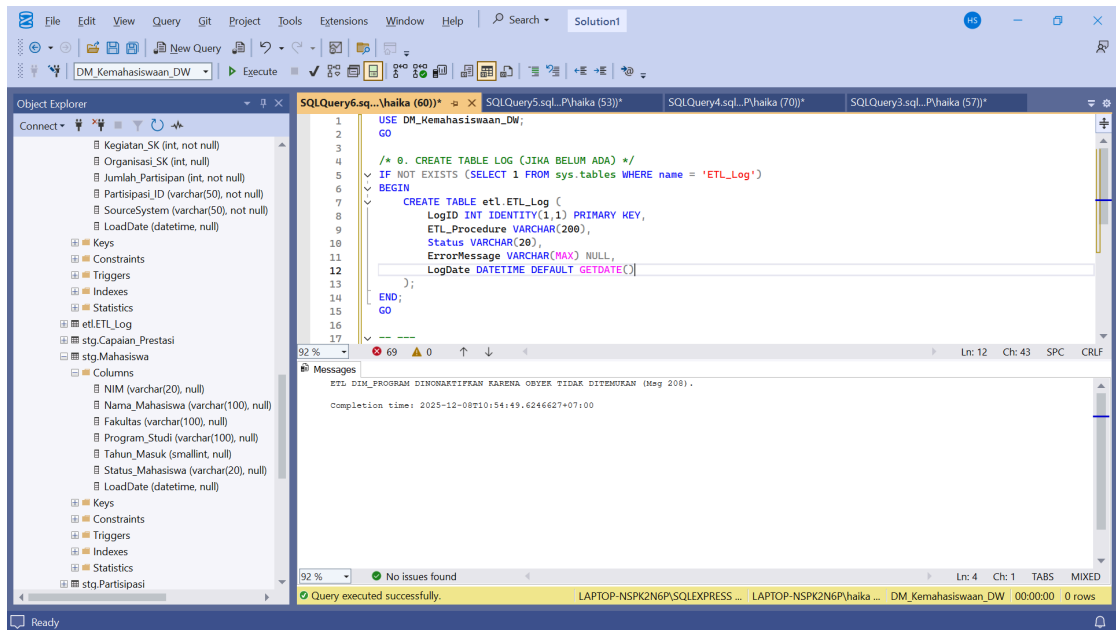
Script terdiri dari dua proses ETL utama dengan strategi Slowly Changing Dimension yang berbeda. ETL pertama memuat data ke Dim_Program menggunakan teknik SCD Type 1, di mana data dari tabel staging stg.Program dipindahkan ke temporary table terlebih dahulu, kemudian menggunakan perintah MERGE untuk melakukan update jika terjadi perubahan

pada ProgramName atau Faculty, atau insert jika record belum ada. Perubahan data akan langsung menimpa record lama tanpa menyimpan histori. ETL kedua memuat data ke Dim_Mahasiswa menggunakan teknik SCD Type 2 yang lebih kompleks, di mana perubahan data tidak menimpa record lama tetapi membuat versi baru. Proses dimulai dengan operasi UPDATE yang menonaktifkan record lama (IsCurrent = 0) dan mengisi ExpiryDate ketika terdeteksi perubahan pada kolom Nama_Mahasiswa, ProgramCode, atau Status_Mahasiswa. Kemudian operasi INSERT memasukkan record baru atau versi terbaru dengan melakukan lookup ke Dim_Program untuk mendapatkan informasi program studi dan fakultas, serta melakukan transformasi data seperti UPPER(TRIM()) pada nama mahasiswa dan ekstraksi tahun dari tanggal pendaftaran. Clause WHERE NOT EXISTS memastikan hanya record yang belum memiliki versi aktif yang akan disisipkan, mencegah duplikasi data.

```
USE DM_Kemahasiswaan_DW;
GO

/* 0. CREATE TABLE LOG (JIKA BELUM ADA) */
IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE name = 'ETL_Log')
BEGIN
    CREATE TABLE etl.ETL_Log (
        LogID INT IDENTITY(1,1) PRIMARY KEY,
        ETL_Procedure VARCHAR(200),
        Status VARCHAR(20),
        ErrorMessage VARCHAR(MAX) NULL,
        LogDate DATETIME DEFAULT GETDATE()
    );
END;
GO

-- ---
-- Karena 'dbo.Dim_Program' Invalid Object Name (Msg 208),
-- Bagian ETL ini dinonaktifkan sementara atau harus merujuk ke skema yang
-- benar.
-- ---
PRINT 'ETL DIM_PROGRAM DINONAKTIFKAN KARENA OBYEK TIDAK
DITEMUKAN (Msg 208).';
-- GO
```



Pada tahap ini dilakukan pemeriksaan awal terhadap keberadaan tabel log untuk proses ETL dalam database DM_Kemahasiswaan_DW dengan menggunakan perintah IF NOT EXISTS. Jika tabel tersebut belum tersedia, maka sistem akan otomatis membuat tabel etl.ETL_Log yang berisi informasi log seperti LogID, nama prosedur ETL, status eksekusi, pesan kesalahan, serta tanggal pencatatan. Tabel ini berfungsi sebagai wadah pencatatan setiap aktivitas ETL, sehingga memungkinkan pengguna untuk melakukan monitoring, audit, dan pelacakan error apabila terjadi kegagalan proses. Setelah pengecekan dan pembuatan tabel log, sistem mengeluarkan pesan bahwa proses ETL untuk Dim_Program dinonaktifkan sementara, karena objek dbo.Dim_Program tidak ditemukan pada database seperti yang ditunjukkan oleh error Invalid Object Name (Msg 208). Pesan tersebut menandakan bahwa modul ETL terkait belum dapat dijalankan sebelum skema atau nama tabel diperbaiki sesuai struktur yang ada.

4. User Manual (Panduan Pengguna)

User Manual disusun untuk memberikan panduan kepada pengguna akhir dalam mengoperasikan sistem Data Mart yang telah dibangun. Dokumen ini berisi instruksi lengkap mengenai cara mengakses, menggunakan, dan memanfaatkan fitur-fitur yang tersedia pada dashboard analisis. Isi User Manual pada sistem ini meliputi:

1) Pendahuluan Pengguna

Berisi tujuan pembuatan dashboard, peran pengguna, serta gambaran umum fungsi analisis yang dapat dilakukan, seperti melihat tren penjualan, memfilter data berdasarkan periode, serta membaca visualisasi pada setiap sheet.

2) Persyaratan Sistem Pengguna

Menjelaskan kebutuhan minimum perangkat yang digunakan user, seperti Browser yang kompatibel (Chrome/Edge), akses internet stabil, dan aplikasi Tableau Viewer / akses Tableau Public

3) Cara Mengakses Dashboard

Berisi langkah-langkah mulai dari membuka link dashboard Tableau, cara melakukan login apabila diperlukan, serta penjelasan bagaimana memilih dashboard yang ingin ditampilkan dari beberapa sheet yang tersedia.

4) Navigasi Menu Dashboard

Menjelaskan setiap elemen yang ada pada dashboard, seperti: Filter kategori, Filter tanggal, Tombol navigasi antar dashboard dan Cara membaca grafik (bar chart, line chart, pie chart)

5) Cara Menggunakan Fitur Utama

User Manual menjelaskan penggunaan fitur analitik, seperti:

- Melakukan filter multi-level
- Drill-down dan drill-up
- Menampilkan detail data pada tooltip
- Melakukan export dashboard ke PDF atau image

6) Contoh Skenario Penggunaan

Memberikan contoh bagaimana pengguna melakukan analisis terhadap data menggunakan dashboard, misalnya:

- Menentukan bulan dengan penjualan tertinggi
- Melihat kontribusi kategori tertentu
- Melakukan perbandingan antar wilayah

7) Troubleshooting Dasar

Berisi solusi awal untuk masalah yang sering terjadi, seperti:

- Dashboard tidak tampil
- Filter tidak berfungsi
- Grafik tidak muncul karena koneksi lambat

5. Operations Manual

Operator Manual ditujukan untuk admin atau pihak teknis yang bertanggung jawab terhadap pemeliharaan sistem Data Mart, termasuk ETL, database, data staging, hingga publikasi dashboard.

Komponen Operasional	Deskripsi	Penanggung Jawab
Data Mart	Data Mart menyimpan data kemahasiswaan untuk analisis: partisipasi kegiatan, dana kegiatan, capaian prestasi, dan penerima beasiswa.	Admin DW
Struktur Utama	4 Fact Table: Partisipasi Kegiatan, Dana Kegiatan, Capaian Prestasi, Beasiswa. 8 Dimensi: Mahasiswa, Kegiatan, Tanggal, Prestasi, Beasiswa, Organisasi, Fakultas, Program Studi.	Admin DW

Sumber Data	Data berasal dari sistem akademik & kemahasiswaan, diambil melalui Staging Area (stg) sebelum di-load ke dimensi/fact.	Admin ETL
ETL Workflow	Dilakukan dengan stored procedure: • usp_Load_Dim_* • usp_Load_Fact_* Urutan eksekusi: Dimensi → Fact.	Admin ETL
Schedule	ETL otomatis melalui SQL Server Agent Job: • Fact: Harian 01:00 • Dimensi: Mingguan.	Operator
Backup Strategy	• Full Backup: Mingguan • Log Backup: Harian/jam. Format file disimpan sesuai tanggal.	Operator Server
Audit & Security	Masking NIM, role-based access (db_viewer, db_operational, dll), dan audit trigger pada Fact.	DBA
Monitoring & Log	Semua proses ETL mencatat status ke tabel etl.ETL_Log (Success/Failed, Error Message).	Admin ETL
Error Handling	Jika error: rollback transaction, simpan error di log, dan kirim notifikasi ke admin.	Admin ETL
Dashboard Deployment	Dashboard Tableau terhubung ke view analitik (vw_*). Update otomatis setelah ETL selesai.	
Recovery	Jika sistem bermasalah: 1. Restore database dari backup 2. Restore log sampai titik waktu tertentu (PITR).	DBA

Bagian Operations Manual menjelaskan prosedur operasional yang harus dijalankan oleh admin atau pihak teknis untuk menjaga keberlangsungan Data Mart kemahasiswaan. Data Mart ini menyimpan data partisipasi kegiatan, capaian prestasi, beasiswa, dan dana kegiatan, sehingga admin data warehouse bertanggung jawab memastikan struktur empat tabel fakta dan delapan tabel dimensi selalu terpelihara dan terhubung dengan baik. Data bersumber dari sistem akademik dan kemahasiswaan, masuk terlebih dahulu ke staging area sebelum dimuat ke Data Mart melalui proses ETL yang dijalankan menggunakan stored procedure, dengan urutan dimensi terlebih dahulu lalu fakta. Seluruh proses ETL dijadwalkan otomatis melalui SQL Server Agent (harian untuk fact, mingguan untuk dimensi) dan dicatat ke tabel log agar setiap keberhasilan maupun kegagalan dapat ditelusuri. Strategi backup juga

diterapkan, yaitu full backup mingguan dan log backup harian atau per jam untuk mendukung pemulihan titik waktu, sedangkan keamanan dijaga melalui masking NIM, role-based access, dan audit trigger pada tabel fakta. Jika terjadi error, transaksi akan di-rollback, pesan dicatat, dan admin diberi notifikasi, sementara publikasi dashboard Tableau dihubungkan dengan view analitik dan diperbarui secara otomatis setelah ETL berjalan. Pemulihan sistem dilakukan dengan merestore database dan log backup sampai titik waktu tertentu, sehingga keseluruhan proses operasional dapat berjalan stabil, terawasi, dan aman.

6. Security Documentation

```
USE DM_Kemahasiswaan_DW;
GO

-- 1. DOKUMENTASI ROLE DAN USER (RBAC)
SELECT '1. ROLE DAN USER' AS [Dokumentasi],
       dp.name AS [Prinsipal Database],
       dp.type_desc AS [Tipe Prinsipal],
       drm.name AS [Anggota Role],
       dp_login.name AS [Login Server Terkait]
FROM sys.database_principals dp
LEFT JOIN sys.database_role_members drm_map ON dp.principal_id =
drm_map.member_principal_id
LEFT JOIN sys.database_principals drm ON drm_map.role_principal_id =
drm.principal_id
LEFT JOIN sys.server_principals dp_login ON dp.sid = dp_login.sid
WHERE dp.type IN ('S', 'U', 'R') -- S=SQL User, U=Windows User, R=Database
Role
ORDER BY 1, 3, 2;
GO

-- 2. DOKUMENTASI PERMISSIONS (GRANT yang diberikan)
SELECT '2. PERMISSIONS GRANTED' AS [Dokumentasi],
       dp.name AS [Prinsipal (User/Role)],
       permission_name AS [Izin Diberikan],
       CASE WHEN major_id = 0 THEN 'DATABASE'
            WHEN major_id > 0 AND class = 1 THEN OBJECT_NAME(major_id)
            WHEN class = 3 THEN 'SCHEMA::' + SCHEMA_NAME(minor_id)
            ELSE 'Lainnya'
       END AS [Objek Target],
       STATE_DESC AS [Status]
FROM sys.database_permissions AS dpms
INNER JOIN sys.database_principals AS dp ON dpms.grantee_principal_id =
dp.principal_id
WHERE dp.name IN ('db_executive', 'db_operational', 'db_viewer',
'db_etl_operator')
ORDER BY dp.name, [Izin Diberikan];
GO
```

File Edit View Query Tools Extensions Window Help Search Solution
Ln 52 Col 52 SPC CRLF

DM_Kemahasiswaan.DW
Execute
SQLQuery1.sql - Paha (54?)
SQLQuery12a - Paha (53?)
SQLQuery13a - Paha (7?)
SQLQuery14a - Paha (7?)
SQLQuery5.sql - Executing...
SQLQuery6.sql - Paha (8?)
SQLQuery7.sql - Paha (6?)

Object Explorer

```

-- 3. DOCUMENTASI DATA MASKING
GO
SELECT '3. DATA MASKING' AS [Dokumentasi],
       t.name AS [Data Tabel],
       c.name AS [Data Kolom Masked],
       c.masking_function AS [Fungsi Masking],
       CASE WHEN EXISTS (SELECT 1 FROM sys.databases.permissions
                        WHERE class = 107 AND permission_name = 'UPDATE'
                        AND grantee_principal_id = dp.principal_id)
       THEN 'GRANTED' ELSE 'DENIED' END AS [Hak UMSMKS Pawan]
FROM sys.master_columns c
INNER JOIN sys.tables AS t ON c.object_id = t.object_id
INNER JOIN sys.databases.principals AS dp ON dp.type = 'R' AND dp.name IN ('db_executor', 'db_operational')
ORDER BY 2, 3, 4;
GO

-- 4. DOCUMENTASI AUDIT TRAIL (Contoh Perbarisan Trigger Audit)
GO
SELECT '4. AUDIT TRAIL (Trigger)' AS [Dokumentasi],
       t.name AS [Data Trigger],
       SELECT NULL AS [parent_id], AS [Tabel Disaudit],
       CASE WHEN IS_TRIGGER_OR_TRIGGERED(t.object_id, 'ExcuteInsertTrigger') = 1 THEN 'UPDATE' ELSE '' END AS
       CASE WHEN IS_TRIGGER_OR_TRIGGERED(t.object_id, 'ExcuteInsertTrigger') = 1 THEN 'INSERT' ELSE '' END AS
       CASE WHEN IS_TRIGGER_OR_TRIGGERED(t.object_id, 'ExcuteDeleteTrigger') = 1 THEN 'DELETE' ELSE '' END AS [Type Aksi],
       t.is_disabled AS [Disabled]
FROM sys.triggers a1
WHERE object_id = parent_id = 'Fact_Capaian_Prestasi'; -- GANTI NAMA TABEL FAKTA ZISBA BERBEDA
GO

```

Results Messages
Ln 52 Col 52 SPC CRLF

Dokumentasi		Principal Database	Type Principal	Anggota Role	Logan Server Terkat
1	ROLE DBA USER	db_accessadmin	DATABASE_ROLE	NULL	NULL
2	ROLE DBA USER	db_accessoperator	DATABASE_ROLE	NULL	NULL
3	ROLE DBA USER	db_datareader	DATABASE_ROLE	NULL	NULL
4	ROLE DBA USER	db_datawriter	DATABASE_ROLE	NULL	NULL
5	ROLE DBA USER	db_ddladmin	DATABASE_ROLE	NULL	NULL
6	ROLE DBA USER	db_denydatareader	DATABASE_ROLE	NULL	NULL
7	ROLE DBA USER	db_denydatawriter	DATABASE_ROLE	NULL	NULL
8	ROLE DBA USER	db_owner	DATABASE_ROLE	NULL	NULL

Dokumentasi		Principal (User/Role)	Is Databank	Objek Target	Status
1	2 PERMISSIONS GRANTED	db_owner	DELETE	NULL	GRANT
2	2 PERMISSIONS GRANTED	db_owner	DELETE	NULL	GRANT
3	2 PERMISSIONS GRANTED	db_owner	EXECUTE	NULL	GRANT
4	2 PERMISSIONS GRANTED	db_owner	INSERT	NULL	GRANT
5	2 PERMISSIONS GRANTED	db_owner	INSERT	NULL	GRANT
6	2 PERMISSIONS GRANTED	db_owner	SELECT	NULL	GRANT
7	2 PERMISSIONS GRANTED	db_owner	SELECT	NULL	GRANT
8	2 PERMISSIONS GRANTED	db_owner	UPDATE	NULL	GRANT

Dokumentasi		Nama Tabel	Nama Kolom Masked	Fungsi Masking	Hak UMSMKS Pawan
1	3 DATA MASKING	Des_Mahasiswa	Email	email()	DENIED
2	3 DATA MASKING	Des_Mahasiswa	Email	email()	DENIED

Query executed successfully.
LAPTOP-N9PKN7M9\SQLSERVER... LAPTOP-N9PKN7M9\haha... DM_Kemahasiswaan.DW 00:00:01 54 rows

Implementasi keamanan pada database DM_Kemahasiswaan_DW dilakukan melalui pendekatan menyeluruh yang mencakup pengelolaan akses berbasis peran (RBAC), masking data sensitif, serta pencatatan aktivitas melalui audit. Tahap awal membentuk empat peran pengguna dengan karakteristik berbeda, yaitu db_executive untuk level pimpinan yang memiliki akses baca penuh termasuk data sensitif yang tidak termasking, db_operational untuk operasional harian dengan hak baca semua data dan hak tulis pada skema staging, db_viewer yang dibatasi hanya pada view analitik tanpa akses terhadap data yang termasking, serta db_etl_operator sebagai akun layanan untuk menjalankan proses ETL dengan hak eksekusi stored procedure. Setiap peran dihubungkan dengan login dan user yang dibuat khusus menggunakan password kuat, dilengkapi aturan keamanan standar melalui CHECK_POLICY. Pemberian hak akses dilakukan secara selektif, misalnya viewer hanya dapat membaca Dim_Date dan view dashboard seperti vw_Activity_Summary sehingga aman untuk konsumsi publik visualisasi. Selain itu, kolom NIM dalam tabel Dim_Student dikategorikan sebagai data sensitif dan diberikan kebijakan data masking menggunakan fungsi partial(4,"XXXX",4), sehingga hanya peran yang memiliki hak UNMASK yaitu executive dan operational yang dapat melihat data asli, sementara viewer akan melihat format tersamarkan. Mekanisme audit diterapkan dengan membuat tabel AuditLog dan trigger pada fact table Fact_Capaian_Prestasi untuk mencatat secara otomatis setiap operasi INSERT, UPDATE, dan DELETE, termasuk nama pengguna, jenis kejadian, objek yang terpengaruh, serta jumlah baris yang berubah. Langkah ini memastikan jejak aktivitas tersimpan sebagai bukti kontrol, memudahkan investigasi, pemantauan integritas data, dan memenuhi kebutuhan tata kelola atau kepatuhan. Melalui desain tersebut, keamanan data mart mahasiswa dapat terjaga, akses terkontrol sesuai peran, data penting terlindungi, dan perubahan dapat ditelusuri secara akurat.

BAB V

PENUTUP

I. Kesimpulan

Seluruh tujuan Misi 3, yaitu Production Deployment, integrasi data Prestasi dan Beasiswa, implementasi keamanan berlapis (DDM & Audit Trail), dan strategi Point-in-Time Recovery, telah berhasil dicapai. Data Mart Kemahasiswaan ITERA kini beroperasi pada lingkungan produksi yang aman, terotomasi, dan siap mendukung pengambilan keputusan strategis oleh pimpinan. Kepatuhan SCD Type 2 dipastikan pada logika ETL Fact Table baru, yang merupakan pencapaian krusial dalam menjaga akurasi historis data.

II. Future Work

Untuk memaksimalkan nilai bisnis Data Mart Kemahasiswaan di masa depan, disarankan untuk melakukan pekerjaan lanjutan berikut:

1. Ekspansi Data Keuangan (Budgeting): Mengintegrasikan Fact_Pengeluaran_Dana dengan detail anggaran vs. realisasi untuk analisis

efisiensi biaya yang lebih mendalam, melengkapi analisis beasiswa yang sudah ada.

2. Advanced Predictive Analytics: Mengembangkan model analitik untuk memprediksi mahasiswa yang berisiko rendah/tinggi dalam mencapai step 6
3. AI prestasi akademik atau non-akademik, memungkinkan intervensi dini.
4. Cloud Optimization: Melakukan studi migrasi Data Mart ke cloud platform (seperti Azure SQL Database atau AWS Redshift) untuk memanfaatkan skalabilitas elastis, mengurangi overhead operasional, dan mendukung akses data yang lebih luas dan terdistribusi.
5. Automated Reporting: Mengimplementasikan Automated Reporting (misalnya, notifikasi email mingguan) untuk pimpinan terkait tren KPI yang signifikan atau anomali data.